TEST CASE SPECIFICATION

for

ResQ

Crisis Coordination Platform for Emergency Response

Version 1.0

Prepared by : 1. Gurnoor Singh Marok (1024170226)
2. Rishi Gupta (1024170043)
3. Tamanna Chadha (1027170237)
4. Kaif Ahmad (1024170255)

Submitted to : Ms. Anushka

November 2, 2026

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope and Sources . . . 3
1.3 How to Read a Test Case . . . 3

2 Summary of Results **4**
2.1 Test Suites . . . 4
2.2 Test Cases by Type . . . 4
2.3 Test Cases by Feature Area . . . 4

3 Test Cases by Feature Area **5**
3.1 Authentication and roles . . . 5
3.2 Request lifecycle . . . 10
3.3 Privacy and location . . . 14
3.4 Resources and inventory . . . 19
3.5 Messaging and notifications . . . 26
3.6 Evidence . . . 28
3.7 Categories and incidents . . . 30
3.8 Organizations . . . 31
3.9 Verification . . . 33
3.10 Cancellation . . . 36
3.11 Real-time (Socket.IO) . . . 38
3.12 Matching . . . 42
3.13 Analytics . . . 45
3.14 AI classification . . . 47
3.15 Seed data and health . . . 50
3.16 End-to-end scenarios . . . 51

4 Traceability **56**
4.1 Endpoints to Test Cases . . . 56
4.2 Roles to Permission Tests . . . 56
4.3 Source Files to Test Cases . . . 57
4.4 SRS Functional Requirements to Test Cases . . . 58

5 Manual Validation Records **60**
5.1 Tickets Processed . . . 60
5.2 Simultaneous-Claim Test . . . 60
5.3 Measured Metrics . . . 60

6 Known Coverage Gaps **61**
6.1 Route Endpoints With No Automated Tests . . . 61
6.2 Endpoints With Happy-Path Only (Missing Negative Tests) . . . 61

1 Introduction

1.1 Purpose

This document lists every automated test case of ResQ, the crisis coordination platform described in the accompanying Software Requirements Specification. For each of the 125 test cases it gives the place of the test in the source code, the source code that the test exercises, the real-world use case it protects, its steps and its expected result. It also links each test case to the functional requirements of the SRS, and it lists the manual validation done in the pilot and the parts of the system that still have no automated test.

1.2 Scope and Sources

The inventory was generated from the repository itself (commit a6e5f2e, 2026-11-01) by a script that parses the test files. It covers the server tests written with Vitest and Supertest and the end-to-end tests written with Playwright. All 125 tests passed in the run on which this document is based. The results are those of one run on one machine, and the document has to be regenerated whenever tests are added.

The location of every test and every item of code under test was checked mechanically: the file exists, the line range lies inside the file, and the named function or route appears within that range. This shows that the reference is correct. It does not show that the test exercises every line of that code. The links to the SRS requirements were prepared by the authors from the titles and use cases of the tests, and are not generated.

1.3 How to Read a Test Case

Every test case is shown in a table with the following rows.

Type, polarity, roles: unit, integration or end-to-end; positive when the test shows that something works or is allowed, negative when it shows that something is correctly rejected; and the user roles the test acts as.

Test location: the test file and the line of the test, followed by the full test name including its enclosing groups.

Code under test: the source files and line ranges the test exercises, with the function or route handler.

Use case, preconditions, steps, expected result: the situation the test protects and what it does and checks.

SRS requirement(s): the functional requirements (FR numbers) of the SRS that the test supports.

Phase, last run: the project phase in which the test was introduced and the result of the latest run.

2 Summary of Results

2.1 Test Suites

Suite	Files	Tests	Passed	Command	Duration
Server (Vitest and Supertest)	21	118	118	cd server && npm test	32.02s
Client end-to-end (Playwright)	3	7	7	cd client && npx playwright test	1.3m (78s)

2.2 Test Cases by Type

Type	Tests	Passed	Failed
Unit	17	17	0
Integration	101	101	0
End-to-end	7	7	0
Total	**125**	**125**	**0**

Of the 125 test cases, 90 are positive and 35 are negative, that is, 35 check that something is correctly refused.

2.3 Test Cases by Feature Area

Feature area	Code	Tests	Positive	Negative	Passed
Authentication and roles	AUTH	15	5	10	15
Request lifecycle	LIFE	12	6	6	12
Privacy and location	PRIV	11	8	3	11
Resources and inventory	RESC	19	11	8	19
Messaging and notifications	COMM	6	6	0	6
Evidence	EVID	3	2	1	3
Categories and incidents	CAT	3	3	0	3
Organizations	ORG	6	4	2	6
Verification	VRFY	6	4	2	6
Cancellation	CNCL	5	5	0	5
Real-time (Socket.IO)	SOCK	10	7	3	10
Matching	MTCH	6	6	0	6
Analytics	ANLT	5	5	0	5
AI classification	AICL	8	8	0	8
Seed data and health	SEED	3	3	0	3
End-to-end scenarios	E2E	7	7	0	7

By the project phase in which they were introduced, the tests are distributed as follows: Phase 1: 37; Phase 2: 31; Phase 2B: 1; Phase 3: 10; Phase 3B: 17; Phase 4: 19; Phase 5: 5; Phase 5 (Location Address Search): 5.

3 Test Cases by Feature Area

3.1 Authentication and roles

Covers credential verification, JWT generation, bcrypt password hashing, session recovery via /api/auth/me, and RBAC middleware (requireAuth and requireRole) blocking unauthorized access to protected end-points.

TC-AUTH-01 should register a new volunteer successfully and return a JWT

Type, polarity, roles	Integration (Vitest and Supertest); Positive; volunteer
Test location	server/tests/auth.test.ts:30 POST /api/auth/register > should register a new volunteer successfully and return a JWT
Code under test	server/src/routes/auth.ts:10-86 POST /api/auth/register server/src/models/User.ts:67-115 userSchema
Use case	A new civilian signs up as a field volunteer to offer emergency disaster assistance on scene.
Preconditions	Target email is unallocated in User collection and database connection is healthy.
Steps	1. Submit POST /api/auth/register with name, email, password, and role “volunteer” 2. Verify HTTP 201 response payload containing valid JWT and sanitized user profile
Expected result	New user document saved with bcrypt hashed password; returns JWT token without exposing password hash.
SRS requirement(s)	FR-2, FR-3
Phase, last run	Phase 1; Pass (2026-11-01)

TC-AUTH-02 should reject registration with role “admin” with 403 Forbidden

Type, polarity, roles	Integration (Vitest and Supertest); Negative; anonymous
Test location	server/tests/auth.test.ts:55 POST /api/auth/register > should reject registration with role “admin” with 403 Forbidden
Code under test	server/src/routes/auth.ts:10-86 POST /api/auth/register
Use case	Prevents untrusted public registrants from escalating privileges to central dispatch administrator.
Preconditions	Public registration endpoint exposed.
Steps	1. Submit POST /api/auth/register attempting to assign role “admin” 2. Verify rejection code and security alert message
Expected result	HTTP 403 Forbidden with security error stating admin accounts cannot be created via public endpoint.
SRS requirement(s)	FR-2
Phase, last run	Phase 1; Pass (2026-11-01)

TC-AUTH-03 should reject registration if email already exists with 409 Conflict

Type, polarity, roles	Integration (Vitest and Supertest); Negative; anonymous
Test location	server/tests/auth.test.ts:71 POST /api/auth/register > should reject registration if email already exists with 409 Conflict
Code under test	server/src/routes/auth.ts:10-86 POST /api/auth/register
Use case	Blocks duplicate account creation to prevent identity collisions among field responders.
Preconditions	An active user account is already registered with the specified email address.
Steps	1. Submit POST /api/auth/register using an existing registered email 2. Assert conflict response status and error message
Expected result	HTTP 409 Conflict with message indicating an account already exists with that email.
SRS requirement(s)	FR-2
Phase, last run	Phase 1; Pass (2026-11-01)

TC-AUTH-04 should reject registration if password is too short

Type, polarity, roles	Integration (Vitest and Supertest); Negative; anonymous
Test location	server/tests/auth.test.ts:90 POST /api/auth/register > should reject registration if password is too short
Code under test	server/src/routes/auth.ts:10-86 POST /api/auth/register
Use case	Enforces minimum credential length requirements to prevent weak passwords on responder accounts.
Preconditions	Registration endpoint active.
Steps	1. Submit POST /api/auth/register with a password under 6 characters 2. Verify input validation failure
Expected result	HTTP 400 Bad Request with password complexity requirements error.
SRS requirement(s)	FR-2
Phase, last run	Phase 1; Pass (2026-11-01)

TC-AUTH-05 should authenticate valid credentials and return a JWT

Type, polarity, roles	Integration (Vitest and Supertest); Positive; volunteer
Test location	server/tests/auth.test.ts:114 POST /api/auth/login > should authenticate valid credentials and return a JWT
Code under test	server/src/routes/auth.ts:88-137 POST /api/auth/login
Use case	A registered responder signs into the dispatch console to view open incident queues.
Preconditions	User registered in MongoDB with valid bcrypt password hash.
Steps	1. Submit POST /api/auth/login with valid email and password 2. Verify response status and parse returned JWT payload
Expected result	HTTP 200 OK with authenticated JWT token containing userId and role claims.
SRS requirement(s)	FR-3
Phase, last run	Phase 1; Pass (2026-11-01)

TC-AUTH-06 should reject login with invalid password

Type, polarity, roles	Integration (Vitest and Supertest); Negative; anonymous
Test location	server/tests/auth.test.ts:126 POST /api/auth/login > should reject login with invalid password
Code under test	server/src/routes/auth.ts:88-137 POST /api/auth/login
Use case	Rejects login attempts when an incorrect password is provided for an existing account.
Preconditions	User account exists with known password.
Steps	1. Submit POST /api/auth/login with valid email but incorrect password 2. Assert unauthorized response code
Expected result	HTTP 401 Unauthorized with generic credential failure message.
SRS requirement(s)	FR-3
Phase, last run	Phase 1; Pass (2026-11-01)

TC-AUTH-07 should reject login with non-existent email

Type, polarity, roles	Integration (Vitest and Supertest); Negative; anonymous
Test location	server/tests/auth.test.ts:136 POST /api/auth/login > should reject login with non-existent email
Code under test	server/src/routes/auth.ts:88-137 POST /api/auth/login
Use case	Prevents credential enumeration and unauthorized session creation for unregistered addresses.
Preconditions	Target email does not exist in MongoDB User collection.
Steps	1. Submit POST /api/auth/login with an unregistered email 2. Assert rejection status code
Expected result	HTTP 401 Unauthorized with invalid email or password message.
SRS requirement(s)	FR-3
Phase, last run	Phase 1; Pass (2026-11-01)

TC-AUTH-08 GET /api/auth/me should return profile when authenticated with valid token

Type, polarity, roles	Integration (Vitest and Supertest); Positive; volunteer
Test location	server/tests/auth.test.ts:176 Middleware: requireAuth & requireRole > GET /api/auth/me should return profile when authenticated with valid token
Code under test	server/src/routes/auth.ts:139-163 GET /api/auth/me server/src/middleware/auth.ts:9-35 requireAuth
Use case	Enables client session restoration on page reload by resolving the active user profile from Bearer token.
Preconditions	Valid JWT signed with server JWT_SECRET.
Steps	1. Send GET /api/auth/me with Authorization: Bearer <token> 2. Verify user profile returned without password hash
Expected result	HTTP 200 OK returning user identity, email, and role.
SRS requirement(s)	FR-3
Phase, last run	Phase 1; Pass (2026-11-01)

TC-AUTH-09 GET /api/auth/me should return 401 when no token is provided

Type, polarity, roles	Integration (Vitest and Supertest); Negative; anonymous
Test location	server/tests/auth.test.ts:186 Middleware: requireAuth & requireRole > GET /api/auth/me should return 401 when no token is provided
Code under test	server/src/middleware/auth.ts:9-35 requireAuth
Use case	Protects authenticated session endpoints from unauthenticated callers.
Preconditions	No Authorization header included in HTTP request.
Steps	1. Send GET /api/auth/me without authorization header 2. Assert rejection status
Expected result	HTTP 401 Unauthorized with token required error.
SRS requirement(s)	FR-3
Phase, last run	Phase 1; Pass (2026-11-01)

TC-AUTH-10 GET /api/auth/me should return 401 when an invalid token is provided

Type, polarity, roles	Integration (Vitest and Supertest); Negative; anonymous
Test location	server/tests/auth.test.ts:192 Middleware: requireAuth & requireRole > GET /api/auth/me should return 401 when an invalid token is provided
Code under test	server/src/middleware/auth.ts:9-35 requireAuth
Use case	Blocks requests presenting expired, altered, or forged tokens.
Preconditions	Request includes an invalid or corrupt token string.
Steps	1. Send GET /api/auth/me with forged Bearer token 2. Assert token verification rejection
Expected result	HTTP 401 Unauthorized with invalid or expired token error.
SRS requirement(s)	FR-3
Phase, last run	Phase 1; Pass (2026-11-01)

TC-AUTH-11 GET /api/admin/overview should reject non-admin with 403 Forbidden

Type, polarity, roles	Integration (Vitest and Supertest); Negative; volunteer
Test location	server/tests/auth.test.ts:201 Middleware: requireAuth & requireRole > GET /api/admin/overview should reject non-admin with 403 Forbidden
Code under test	server/src/middleware/auth.ts:37-53 requireRole
Use case	Enforces role boundary so field volunteers cannot view administrative overview controls.
Preconditions	User authenticated with role “volunteer”.
Steps	1. Send GET /api/admin/overview with volunteer JWT 2. Assert RBAC role guard rejection
Expected result	HTTP 403 Forbidden with insufficient privileges error.
SRS requirement(s)	FR-4
Phase, last run	Phase 1; Pass (2026-11-01)

TC-AUTH-12 GET /api/admin/overview should permit admin user with 200 OK

Type, polarity, roles	Integration (Vitest and Supertest); Positive; admin
Test location	server/tests/auth.test.ts:210 Middleware: requireAuth & requireRole > GET /api/admin/overview should permit admin user with 200 OK
Code under test	server/src/middleware/auth.ts:37-53 requireRole
Use case	Grants central dispatch administrators access to administrative management endpoints.
Preconditions	User authenticated with role “admin”.
Steps	1. Send GET /api/admin/overview with admin JWT 2. Verify access granted and payload structure
Expected result	HTTP 200 OK with admin overview statistics.
SRS requirement(s)	FR-4
Phase, last run	Phase 1; Pass (2026-11-01)

TC-AUTH-13 should create a valid user with a recognized role

Type, polarity, roles	Unit (Vitest); Positive; automated / internal
Test location	server/tests/models.test.ts:47 User Model > should create a valid user with a recognized role
Code under test	server/src/models/User.ts:67-115 userSchema
Use case	Validates that User documents can be created across all recognized system roles (admin, volunteer, affected_person, etc.).
Preconditions	Mongoose connection established.
Steps	1. Instantiate User model with valid email, passwordHash, and role “volunteer” 2. Save document to database 3. Assert saved document has valid _id and matches role
Expected result	User document successfully persisted in database.
SRS requirement(s)	FR-2
Phase, last run	Phase 1; Pass (2026-11-01)

TC-AUTH-14 should reject invalid user roles

Type, polarity, roles	Unit (Vitest); Negative; automated / internal
Test location	server/tests/models.test.ts:61 User Model > should reject invalid user roles
Code under test	server/src/models/User.ts:67-115 userSchema (Role Enum Validator)
Use case	Enforces Mongoose enum validation preventing users from being created with arbitrary or invalid role strings.
Preconditions	Mongoose User schema initialized.
Steps	1. Attempt to create User with role “super_hero” 2. Assert save operation throws Mongoose ValidationError on role field
Expected result	ValidationError thrown rejecting unrecognized role.
SRS requirement(s)	FR-2, FR-4
Phase, last run	Phase 1; Pass (2026-11-01)

TC-AUTH-15 should enforce unique email addresses

Type, polarity, roles	Unit (Vitest); Negative; automated / internal
Test location	server/tests/models.test.ts:73 User Model > should enforce unique email addresses
Code under test	server/src/models/User.ts:67-115 userSchema (Unique Email Index)
Use case	Prevents duplicate account creation by enforcing unique database indexes on user email addresses.
Preconditions	User indexes synchronized with MongoDB.
Steps	1. Create and save user with email test@example.com 2. Attempt to save second user with identical email test@example.com 3. Assert MongoDB duplicate key error (code 11000) is thrown
Expected result	Duplicate key error thrown preventing duplicate emails.
SRS requirement(s)	FR-2
Phase, last run	Phase 1; Pass (2026-11-01)

3.2 Request lifecycle

Governs the strict linear progression of emergency requests across the 5 canonical phases (unverified -> verified -> in_progress -> resolved -> closed), disallowing out-of-sequence jumps, retrogressions, or unauthorized status alterations.

TC-LIFE-01 should complete the full happy path: unverified -> verified -> in_progress -> resolved -> closed

Type, polarity, roles	Integration (Vitest and Supertest); Positive; admin, volunteer, affected_person
Test location	server/tests/lifecycle.test.ts:81 Emergency Request Lifecycle Endpoints > should complete the full happy path: unverified -> verified -> in_progress -> resolved -> closed
Code under test	server/src/routes/requests.ts:272-422 PATCH /api/requests/:id/verify server/src/routes/requests.ts:454-546 PATCH /api/requests/:id/claim server/src/routes/requests.ts:547-646 PATCH /api/requests/:id/resolve server/src/routes/requests.ts:647-724 PATCH /api/requests/:id/confirm
Use case	A crisis call is verified by admin, claimed by volunteer, resolved on scene, and confirmed closed by victim.
Preconditions	Admin, volunteer, and citizen accounts seeded in database.
Steps	1. Citizen submits request (status: unverified) 2. Admin verifies request via PATCH /api/requests/:id/verify (status: verified) 3. Volunteer claims request via PATCH /api/requests/:id/claim (status: in_progress) 4. Volunteer marks resolved via PATCH /api/requests/:id/resolve (status: resolved) 5. Citizen confirms via PATCH /api/requests/:id/confirm (status: closed)
Expected result	All 5 transitions succeed in strict sequence, updating status and appending audit log in statusHistory.
SRS requirement(s)	FR-9, FR-10, FR-12, FR-13, FR-14, FR-16
Phase, last run	Phase 1; Pass (2026-11-01)

TC-LIFE-02 should reject skipping stages in lifecycle order

Type, polarity, roles	Integration (Vitest and Supertest); Negative; volunteer
Test location	server/tests/lifecycle.test.ts:146 Emergency Request Lifecycle Endpoints > should reject skipping stages in lifecycle order
Code under test	server/src/routes/requests.ts:454-546 PATCH /api/requests/:id/claim
Use case	Prevents responders from claiming raw unverified requests or jumping straight to resolved.
Preconditions	Request exists in state “unverified”.
Steps	1. Volunteer attempts to claim unverified request via PATCH /api/requests/:id/claim 2. Assert invalid state transition rejection
Expected result	HTTP 400 Bad Request indicating request must be in “verified” state before claiming.
SRS requirement(s)	FR-9
Phase, last run	Phase 1; Pass (2026-11-01)

TC-LIFE-03 should reject reversing or modifying completed lifecycle steps

Type, polarity, roles	Integration (Vitest and Supertest); Negative; volunteer
Test location	server/tests/lifecycle.test.ts:177 Emergency Request Lifecycle Endpoints > should reject reversing or modifying completed lifecycle steps
Code under test	server/src/routes/requests.ts:454-546 PATCH /api/requests/:id/claim server/src/routes/requests.ts:547-646 PATCH /api/requests/:id/resolve
Use case	Guarantees lifecycle immutability once a request has reached terminal state closed.
Preconditions	Request reached terminal status “closed”.
Steps	1. Responder attempts to claim or resolve an already-closed request 2. Assert transition rejection
Expected result	HTTP 400 Bad Request indicating terminal status cannot be modified.
SRS requirement(s)	FR-9
Phase, last run	Phase 1; Pass (2026-11-01)

TC-LIFE-04 should enforce role-based access on transitions

Type, polarity, roles	Integration (Vitest and Supertest); Negative; affected person, volunteer
Test location	server/tests/lifecycle.test.ts:194 Emergency Request Lifecycle Endpoints > should enforce role-based access on transitions
Code under test	server/src/routes/requests.ts:272-422 PATCH /api/requests/:id/verify server/src/routes/requests.ts:547-646 PATCH /api/requests/:id/resolve
Use case	Prevents citizens from verifying tickets or unrelated volunteers from resolving tickets claimed by others.
Preconditions	Request in verified state.
Steps	1. Citizen attempts to verify ticket 2. Unrelated volunteer attempts to resolve ticket claimed by another responder
Expected result	HTTP 403 Forbidden on both unauthorized lifecycle actions.
SRS requirement(s)	FR-4, FR-10, FR-12
Phase, last run	Phase 1; Pass (2026-11-01)

TC-LIFE-05 should verify anonymous requester via contact phone and forbid responder force-closing

Type, polarity, roles	Integration (Vitest and Supertest); Positive; anonymous, volunteer
Test location	server/tests/lifecycle.test.ts:269 Emergency Request Lifecycle Endpoints > should verify anonymous requester via contact phone and forbid responder force-closing
Code under test	server/src/routes/requests.ts:647-724 PATCH /api/requests/:id/confirm
Use case	Allows anonymous disaster victim to confirm resolution with phone verification while blocking responders from closing.
Preconditions	Anonymous request resolved with contactPhone registered.
Steps	1. Responder attempts to confirm close (blocked with 403) 2. Anonymous caller submits PATCH /api/requests/:id/confirm with matching contactPhone
Expected result	Responder blocked with 403; anonymous requester confirmed with phone transitions ticket to closed.
SRS requirement(s)	FR-14
Phase, last run	Phase 1; Pass (2026-11-01)

TC-LIFE-06 should allow anonymous bystander submissions with contact phone

Type, polarity, roles	Unit (Vitest); Positive; public / unauthenticated
Test location	server/tests/models.test.ts:136 Request Model > should allow anonymous bystander submissions with contact phone
Code under test	server/src/models/Request.ts:161-245 requestSchema (Anonymous Submissions)
Use case	Ensures bystanders can submit emergency requests without logging in, providing an emergency callback phone number.
Preconditions	Request schema loaded.
Steps	1. Create Request without requester ID, containing contactPhone “+919876543210” 2. Save document to database 3. Assert document is saved successfully without validation error
Expected result	Anonymous bystander request saved with contactPhone.
SRS requirement(s)	FR-1, FR-5
Phase, last run	Phase 1; Pass (2026-11-01)

TC-LIFE-07 should reject invalid category

Type, polarity, roles	Unit (Vitest); Negative; automated / internal
Test location	server/tests/models.test.ts:151 Request Model > should reject invalid category
Code under test	server/src/models/Request.ts:161-245 requestSchema (Category Validator)
Use case	Protects database schema integrity by rejecting unrecognized emergency category strings in Request model validation.
Preconditions	Request schema loaded.
Steps	1. Instantiate Request with category “invalid_cat” 2. Assert validation throws error on category enum constraint
Expected result	Mongoose ValidationError thrown for invalid category.
SRS requirement(s)	FR-7
Phase, last run	Phase 1; Pass (2026-11-01)

TC-LIFE-08 should accept a public emergency request and return requestId and confirmation

Type, polarity, roles	Integration (Vitest and Supertest); Positive; public / affected person
Test location	server/tests/requests.test.ts:32 POST /api/requests (Emergency Request Submission) > should accept a public emergency request and return requestId and confirmation
Code under test	server/src/routes/requests.ts:104-179 POST /api/requests (Emergency Request Submission)
Use case	Allows citizens in distress to submit emergency relief requests anonymously or with phone contact, generating a tracking code.
Preconditions	Emergency categories and database indexes initialized.
Steps	1. Send POST /api/requests with category, title, description, and precise coordinates 2. Assert response status 201 Created 3. Verify formatted requestId (REQ-YYYY-NNNNNN) and confirmationCode are returned
Expected result	HTTP 201 Created with persisted unverified ticket and tracking confirmation code.
SRS requirement(s)	FR-1, FR-5, FR-6
Phase, last run	Phase 1; Pass (2026-11-01)

TC-LIFE-09 should reject submission with missing category

Type, polarity, roles	Integration (Vitest and Supertest); Negative; public / affected person
Test location	server/tests/requests.test.ts:54 POST /api/requests (Emergency Request Submission) > should reject submission with missing category
Code under test	server/src/routes/requests.ts:104-125 POST /api/requests (Payload Validation)
Use case	Protects intake integrity by rejecting emergency request submissions that fail to provide an emergency category.
Preconditions	MongoDB test server running.
Steps	1. Send POST /api/requests with title and coordinates but omitting category 2. Assert response status 400 Bad Request 3. Assert error payload contains validation message for category
Expected result	HTTP 400 Bad Request with field validation failure.
SRS requirement(s)	FR-5
Phase, last run	Phase 1; Pass (2026-11-01)

TC-LIFE-10 should reject submission with invalid category

Type, polarity, roles	Integration (Vitest and Supertest); Negative; public / affected person
Test location	server/tests/requests.test.ts:65 POST /api/requests (Emergency Request Submission) > should reject submission with invalid category
Code under test	server/src/routes/requests.ts:104-125 POST /api/requests (Category Taxonomy Validation)
Use case	Prevents garbage intake by rejecting emergency requests specifying unknown or unrecognized disaster category strings.
Preconditions	Recognized categories loaded in database.
Steps	1. Send POST /api/requests with category set to “alien invasion” 2. Assert response status 400 Bad Request
Expected result	HTTP 400 Bad Request with invalid category notification.
SRS requirement(s)	FR-7
Phase, last run	Phase 1; Pass (2026-11-01)

TC-LIFE-11 should filter requests by status

Type, polarity, roles	Integration (Vitest and Supertest); Positive; public / unauthenticated
Test location	server/tests/requests.test.ts:123 GET /api/requests (Public Requests List) > should filter requests by status
Code under test	server/src/routes/requests.ts:181-212 GET /api/requests (Query Parameter Status Filter)
Use case	Allows dispatchers, volunteers, and the public to filter request listings by operational status (e.g., unverified vs. in_progress).
Preconditions	Requests with mixed statuses seeded in test database.
Steps	1. Send GET /api/requests?status=in_progress 2. Assert response status 200 OK 3. Verify all returned items match the requested status
Expected result	HTTP 200 OK with accurately filtered array of requests.
SRS requirement(s)	FR-37
Phase, last run	Phase 1; Pass (2026-11-01)

TC-LIFE-12 manually executes the full Phase 1 demo scenario: submit -> verify -> claim -> resolve -> confirm

Type, polarity, roles	Integration (Vitest and Supertest); Positive; affected person, admin, volunteer
Test location	server/tests/seed.test.ts:112 Seed Script & End-to-End Phase 1 Integration Validation > manually executes the full Phase 1 demo scenario: submit -> verify -> claim -> resolve -> confirm
Code under test	server/src/routes/requests.ts:104-750 Request Lifecycle Handlers (submit, verify, claim, resolve, confirm)
Use case	Executes the entire end-to-end REST lifecycle sequence across multiple actors from intake to final citizen confirmation.
Preconditions	Seeded test database.
Steps	1. Citizen submits emergency ticket via POST /api/requests 2. Admin verifies ticket via PATCH /api/requests/:id/verify 3. Volunteer claims ticket via PATCH /api/requests/:id/claim 4. Volunteer resolves ticket via PATCH /api/requests/:id/resolve 5. Citizen confirms satisfaction via PATCH /api/requests/:id/confirm 6. Assert ticket reaches closed status
Expected result	Ticket smoothly transitions through all 5 lifecycle phases to closed.
SRS requirement(s)	FR-6, FR-9, FR-12, FR-13, FR-14
Phase, last run	Phase 1; Pass (2026-11-01)

3.3 Privacy and location

Guarantees victim privacy by ensuring public and unassigned queries only expose fuzzed coordinates (200m–400m random displacement) with physical street addresses redacted, while permitting authorized responders and dispatchers to access precise coordinates.

TC-PRIV-01 should return 400 when query parameter “q” is missing or empty

Type, polarity, roles	Integration (Vitest and Supertest); Negative; public / unauthenticated
Test location	server/tests/geocode.test.ts:11 GET /api/geocode (OSM Nominatim Proxy) > should return 400 when query parameter “q” is missing or empty
Code under test	server/src/routes/geocode.ts:56-65 GET /api/geocode (Query Validation)
Use case	Prevents unnecessary external Nominatim proxy requests and rejects empty address lookups when a client omits the search text query.
Preconditions	Geocode router is mounted at /api/geocode with rate limiting and in-memory cache.
Steps	1. Send GET /api/geocode with missing query parameter 2. Send GET /api/geocode?q= 3. Assert response status is 400 and contains validation error message
Expected result	HTTP 400 Bad Request with an informative error stating query parameter “q” is required.
SRS requirement(s)	FR-5
Phase, last run	Phase 5 (Location Address Search); Pass (2026-11-01)

TC-PRIV-02 should return 400 when query parameter “q” is only whitespace

Type, polarity, roles	Integration (Vitest and Supertest); Negative; public / unauthenticated
Test location	server/tests/geocode.test.ts:18 GET /api/geocode (OSM Nominatim Proxy) > should return 400 when query parameter “q” is only whitespace
Code under test	server/src/routes/geocode.ts:56-65 GET /api/geocode (Whitespace Query Validation)
Use case	Rejects address queries containing only whitespace characters to protect upstream OSM Nominatim API quota.
Preconditions	Geocode router is mounted at /api/geocode.
Steps	1. Send GET /api/geocode?q=%20%20%20 2. Assert response status is 400 Bad Request
Expected result	HTTP 400 Bad Request indicating that non-empty text is required.
SRS requirement(s)	FR-5
Phase, last run	Phase 5 (Location Address Search); Pass (2026-11-01)

TC-PRIV-03 should return 200 with matching candidates for a real query and cache subsequent queries

Type, polarity, roles	Integration (Vitest and Supertest); Positive; public / unauthenticated
Test location	server/tests/geocode.test.ts:24 GET /api/geocode (OSM Nominatim Proxy) > should return 200 with matching candidates for a real query and cache subsequent queries
Code under test	server/src/routes/geocode.ts:66-159 GET /api/geocode (Nominatim Proxy & In-Memory Cache)
Use case	Provides human-readable address lookups with latitude and longitude coordinates through a server-side cached Nominatim proxy.
Preconditions	OSM Nominatim API endpoint is reachable or cached in memory.
Steps	1. Send GET /api/geocode?q=Thapar+University 2. Assert HTTP 200 with candidates array containing display_name, lat, and lng 3. Send identical request immediately and verify cached response without secondary upstream call
Expected result	HTTP 200 with candidates array and sub-second cached response on repeat query.
SRS requirement(s)	FR-5
Phase, last run	Phase 5 (Location Address Search); Pass (2026-11-01)

TC-PRIV-04 should auto-generate a formatted requestId and auto-fuzz public coordinates

Type, polarity, roles	Unit (Vitest); Positive; automated / internal
Test location	server/tests/models.test.ts:93 Request Model > should auto-generate a formatted requestId and auto-fuzz public coordinates
Code under test	server/src/models/Request.ts:247-277 requestSchema pre-save hook
Use case	Verifies the pre-save hook on the Request model automatically generates REQ-YYYY-NNNNNN IDs and fuzzed public coordinates.
Preconditions	Request model loaded.
Steps	1. Create Request with precise coordinates [75.85, 30.90] 2. Save document without explicit requestId or publicCoordinates 3. Assert requestId matches regex ^REQ-\d{4}-\d{6}$ 4. Assert publicCoordinates are offset between 200m and 400m from preciseCoordinates
Expected result	Auto-generated formatted ID and fuzzed public coordinates on document save.
SRS requirement(s)	FR-6, FR-17
Phase, last run	Phase 1; Pass (2026-11-01)

TC-PRIV-05 should calculate fuzzed coordinates helper directly within 200m to 400m

Type, polarity, roles	Unit (Vitest); Positive; automated / internal
Test location	server/tests/models.test.ts:127 Request Model > should calculate fuzzed coordinates helper directly within 200m to 400m
Code under test	server/src/models/Request.ts:71-98 calculateFuzzedCoordinates
Use case	Directly tests the coordinate obfuscation helper to verify the displacement distance is mathematically constrained between 200m and 400m.
Preconditions	calculateFuzzedCoordinates utility loaded.
Steps	1. Invoke calculateFuzzedCoordinates([75.85, 30.90]) 2. Calculate distance between original and fuzzed coordinates using Haversine formula 3. Assert distance is >= 0.2km and <= 0.4km
Expected result	Displacement is strictly between 200 and 400 meters.
SRS requirement(s)	FR-17
Phase, last run	Phase 1; Pass (2026-11-01)

TC-PRIV-06 should reject submission with invalid coordinates

Type, polarity, roles	Integration (Vitest and Supertest); Negative; public / affected person
Test location	server/tests/requests.test.ts:77 POST /api/requests (Emergency Request Submission) > should reject submission with invalid coordinates
Code under test	server/src/routes/requests.ts:104-135 POST /api/requests (Geographic Coordinate Range Check)
Use case	Rejects submissions with invalid latitude (outside [-90, 90]) or longitude (outside [-180, 180]) to prevent geospatial query corruption.
Preconditions	MongoDB test database connected.
Steps	1. Send POST /api/requests with latitude: 200 and longitude: -300 2. Assert response status 400 Bad Request 3. Assert validation failure message for coordinates
Expected result	HTTP 400 Bad Request indicating invalid geographical coordinates.
SRS requirement(s)	FR-5
Phase, last run	Phase 1; Pass (2026-11-01)

TC-PRIV-07 should NEVER leak precise coordinates in public list

Type, polarity, roles	Integration (Vitest and Supertest); Positive; public / unauthenticated
Test location	server/tests/requests.test.ts:109 GET /api/requests (Public Requests List) > should NEVER leak precise coordinates in public list
Code under test	server/src/routes/requests.ts:181-212 GET /api/requests (Public Requests List Sanitization) server/src/utils/sanitize.ts:6-24 sanitizeRequest
Use case	Guarantees victim privacy by ensuring public listing endpoints fuzz coordinates and redact precise GPS points and phone numbers.
Preconditions	Test requests created with precise coordinates.
Steps	1. Send unauthenticated GET /api/requests 2. Assert response status 200 OK 3. Inspect all returned items to ensure location.preciseCoordinates and contactPhone are undefined 4. Assert location.publicCoordinates contains fuzzed offsets
Expected result	HTTP 200 OK with strict redaction of victim coordinates and contact info.
SRS requirement(s)	FR-18
Phase, last run	Phase 1; Pass (2026-11-01)

TC-PRIV-08 should hide precise coordinates from unauthenticated public request

Type, polarity, roles	Integration (Vitest and Supertest); Positive; public / unauthenticated
Test location	server/tests/requests.test.ts:181 GET /api/requests/:id (Privacy Rules for Single Request) > should hide precise coordinates from unauthenticated public request
Code under test	server/src/routes/requests.ts:214-270 GET /api/requests/:id (Unauthenticated Redaction) server/src/utils/sanitize.ts:6-24 sanitizeRequest
Use case	Prevents unauthenticated public visitors from viewing exact victim coordinates when opening a single request by ID.
Preconditions	Request document seeded with precise location.
Steps	1. Send unauthenticated GET /api/requests/:id 2. Assert response status 200 OK 3. Verify preciseCoordinates and contactPhone are absent 4. Verify publicCoordinates are present
Expected result	HTTP 200 OK with coordinates fuzzed to public level.
SRS requirement(s)	FR-18
Phase, last run	Phase 1; Pass (2026-11-01)

TC-PRIV-09 should hide precise coordinates from unrelated volunteer

Type, polarity, roles	Integration (Vitest and Supertest); Positive; volunteer (unassigned)
Test location	server/tests/requests.test.ts:191 GET /api/requests/:id (Privacy Rules for Single Request) > should hide precise coordinates from unrelated volunteer
Code under test	server/src/routes/requests.ts:214-270 GET /api/requests/:id (Unassigned Volunteer Sanitization) server/src/utils/sanitize.ts:6-24 sanitizeRequest
Use case	Prevents authenticated volunteers who have not claimed the ticket from accessing the victim exact physical location.
Preconditions	Request document created; volunteer authenticated with JWT.
Steps	1. Send GET /api/requests/:id with Bearer token of an unassigned volunteer 2. Assert response status 200 OK 3. Confirm preciseCoordinates are stripped from response body
Expected result	HTTP 200 OK with preciseCoordinates hidden from unrelated volunteers.
SRS requirement(s)	FR-18
Phase, last run	Phase 1; Pass (2026-11-01)

TC-PRIV-10 should reveal precise coordinates to administrator

Type, polarity, roles	Integration (Vitest and Supertest); Positive; admin
Test location	server/tests/requests.test.ts:201 GET /api/requests/:id (Privacy Rules for Single Request) > should reveal precise coordinates to administrator
Code under test	server/src/routes/requests.ts:214-270 GET /api/requests/:id (Admin Full Clearance)
Use case	Enables crisis dispatchers and administrators to inspect precise GPS coordinates and victim phone for routing and dispatch.
Preconditions	Admin user authenticated with valid JWT; request seeded.
Steps	1. Send GET /api/requests/:id with Bearer token of an administrator 2. Assert response status 200 OK 3. Assert location.preciseCoordinates contains exact latitude and longitude numbers
Expected result	HTTP 200 OK with full un-redacted location data exposed to admin.
SRS requirement(s)	FR-18
Phase, last run	Phase 1; Pass (2026-11-01)

TC-PRIV-11 should reveal precise coordinates to claimant

Type, polarity, roles	Integration (Vitest and Supertest); Positive; volunteer (claimant)
Test location	server/tests/requests.test.ts:212 GET /api/requests/:id (Privacy Rules for Single Request) > should reveal precise coordinates to claimant
Code under test	server/src/routes/requests.ts:214-270 GET /api/requests/:id (Claimant Clearance)
Use case	Reveals precise GPS coordinates to the specific volunteer who claimed the emergency ticket so they can navigate to the scene.
Preconditions	Request claimed by volunteer; volunteer token generated.
Steps	1. Send GET /api/requests/:id with Bearer token of the active claimant volunteer 2. Assert response status 200 OK 3. Assert location.preciseCoordinates is present and matches stored data
Expected result	HTTP 200 OK revealing full precise coordinates to the assigned claimant.
SRS requirement(s)	FR-18
Phase, last run	Phase 1; Pass (2026-11-01)

3.4 Resources and inventory

Manages resource catalog registration, organization depot inventory tracking, stock reservations upon claiming, itemized requirement attachments, and stock releases upon resolution or cancellation.

TC-RESC-01 should automatically create a RequestAssignment when a verified request is claimed

Type, polarity, roles	Integration (Vitest and Supertest); Positive; volunteer
Test location	server/tests/allocation.test.ts:54 Request Claim & RequestAssignment Integration > should automatically create a RequestAssignment when a verified request is claimed
Code under test	server/src/routes/requests.ts:454-546 PATCH /api/requests/:id/claim server/src/models/RequestAssignment.ts:22-38 RequestAssignment
Use case	When a field responder claims an emergency ticket, an auditable RequestAssignment record is created automatically.
Preconditions	Request verified in database.
Steps	1. Volunteer claims request via PATCH /api/requests/:id/claim 2. Query RequestAssignment collection for created assignment
Expected result	Request status in progress; RequestAssignment created with status assigned and responder details.
SRS requirement(s)	FR-12
Phase, last run	Phase 2; Pass (2026-11-01)

TC-RESC-02 should correctly transition from unallocated -> partially_allocated -> fully_allocated across split contributions

Type, polarity, roles	Integration (Vitest and Supertest); Positive; resource provider
Test location	server/tests/allocation.test.ts:135 Resource Allocation: Partial vs. Full Fulfillment > should correctly transition from unallocated -> partially allocated -> fully allocated across split contributions
Code under test	server/src/routes/requests.ts:1104-1271 POST /api/requests/:id/allocate server/src/models/RequestResource.ts:21-43 RequestResource
Use case	Multiple relief depots fulfill a large hospital supply call in partial batches until 100% fulfilled.
Preconditions	Request created needing 100 water crates; Depots A and B hold stock.
Steps	1. Depot A allocates 40 crates (status transitions to partially_allocated) 2. Depot B allocates remaining 60 crates (status transitions to fully_allocated)
Expected result	RequestResource tracking accurately transitions through partial and full fulfillment states.
SRS requirement(s)	FR-24, FR-26
Phase, last run	Phase 2; Pass (2026-11-01)

TC-RESC-03 should reject allocation when requested quantity exceeds available inventory in stock

Type, polarity, roles	Integration (Vitest and Supertest); Negative; resource provider
Test location	server/tests/allocation.test.ts:318 Resource Allocation: Partial vs. Full Fulfillment > should reject allocation when requested quantity exceeds available inventory in stock
Code under test	server/src/routes/requests.ts:1104-1271 POST /api/requests/:id/allocate
Use case	Prevents allocation commitments when depot available stock cannot fulfill the promised quantity.
Preconditions	Depot stock has available quantity 10; Request needs 50.
Steps	1. Depot attempts to allocate 50 units 2. Assert inventory balance check rejection
Expected result	HTTP 400 Bad Request indicating allocation quantity exceeds depot stock.
SRS requirement(s)	FR-24
Phase, last run	Phase 2; Pass (2026-11-01)

TC-RESC-04 should reject allocation when responder has no registered inventory for that resource

Type, polarity, roles	Integration (Vitest and Supertest); Negative; resource provider
Test location	server/tests/allocation.test.ts:370 Resource Allocation: Partial vs. Full Fulfillment > should reject allocation when responder has no registered inventory for that resource
Code under test	server/src/routes/requests.ts:1104-1271 POST /api/requests/:id/allocate
Use case	Prevents phantom fulfillment by disallowing suppliers from allocating goods they do not stock.
Preconditions	Provider holds no inventory record for requested catalog item.
Steps	1. Provider attempts to allocate unstocked resource 2. Assert inventory existence rejection
Expected result	HTTP 400 Bad Request indicating provider has no registered inventory for this resource.
SRS requirement(s)	FR-24
Phase, last run	Phase 2; Pass (2026-11-01)

TC-RESC-05 should reject allocation from unauthorized roles: volunteer and admin receive 403 Forbidden

Type, polarity, roles	Integration (Vitest and Supertest); Negative; volunteer, admin
Test location	server/tests/allocation.test.ts:408 Resource Allocation: Partial vs. Full Fulfillment > should reject allocation from unauthorized roles: volunteer and admin receive 403 Forbidden
Code under test	server/src/routes/requests.ts:1104-1271 POST /api/requests/:id/allocate
Use case	Restricts material stock allocations strictly to authorized resource providers and depot coordinators.
Preconditions	Request with pending supply requirements.
Steps	1. Volunteer and admin submit POST /api/requests/:id/allocate 2. Assert RBAC guard rejection
Expected result	HTTP 403 Forbidden for volunteer and admin roles.
SRS requirement(s)	FR-24, FR-4
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-RESC-06 should allow a citizen / reporter to specify itemized resource needs without requiring mandatory prior authentication

Type, polarity, roles	Integration (Vitest and Supertest); Positive; anonymous
Test location	server/tests/allocation.test.ts:464 Resource Allocation: Partial vs. Full Fulfillment > should allow a citizen / reporter to specify itemized resource needs without requiring mandatory prior authentication
Code under test	server/src/routes/requests.ts:104-180 POST /api/requests server/src/routes/requests.ts:935-1076 POST /api/requests/:id/resources
Use case	An anonymous disaster victim attaches itemized supply needs (e.g. insulin, clean water) when filing an intake report.
Preconditions	Catalog resources seeded; Public intake open.
Steps	1. Anonymous user submits POST /api/requests with initial supply line items 2. Verify RequestResource records created linked to request
Expected result	HTTP 201 Created with itemized resource requirements attached to ticket.
SRS requirement(s)	FR-5, FR-1
Phase, last run	Phase 2; Pass (2026-11-01)

TC-RESC-07 should reject an unrelated authenticated user and an unauthenticated stranger

Type, polarity, roles	Integration (Vitest and Supertest); Negative; affected person, anonymous
Test location	server/tests/allocation.test.ts:511 Resource Allocation: Partial vs. Full Fulfillment > Resource Requirement Authorization on Requests (POST /api/requests/:id/resources) > should reject an unrelated authenticated user and an unauthenticated stranger
Code under test	server/src/routes/requests.ts:935-1076 POST /api/requests/:id/resources
Use case	Blocks random strangers and unrelated users from tampering with another victim's supply requirements list.
Preconditions	Request created by User A.
Steps	1. Unrelated User B attempts to append supply requirement to Request A 2. Unauthenticated stranger without phone attempts append
Expected result	HTTP 403 Forbidden on unauthorized requirement modifications.
SRS requirement(s)	FR-23
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-RESC-08 should allow the authenticated original requester to append resource requirements

Type, polarity, roles	Integration (Vitest and Supertest); Positive; affected person
Test location	server/tests/allocation.test.ts:582 Resource Allocation: Partial vs. Full Fulfillment > Resource Requirement Authorization on Requests (POST /api/requests/:id/resources) > should allow the authenticated original requester to append resource requirements
Code under test	server/src/routes/requests.ts:935-1076 POST /api/requests/:id/resources
Use case	A displaced family updates their intake ticket to add emergency baby formula after initial submission.
Preconditions	Ticket created by authenticated citizen.
Steps	1. Creator submits POST /api/requests/:id/resources with baby food item 2. Verify requirement appended
Expected result	HTTP 201 Created with new RequestResource item added.
SRS requirement(s)	FR-23
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-RESC-09 should allow an anonymous requester with matching contactPhone to append requirements

Type, polarity, roles	Integration (Vitest and Supertest); Positive; anonymous
Test location	server/tests/allocation.test.ts:619 Resource Allocation: Partial vs. Full Fulfillment > Resource Requirement Authorization on Requests (POST /api/requests/:id/resources) > should allow an anonymous requester with matching contactPhone to append requirements
Code under test	server/src/routes/requests.ts:935-1076 POST /api/requests/:id/resources
Use case	Bystander who submitted an emergency ticket anonymously appends additional needs using their registered phone number.
Preconditions	Ticket submitted anonymously with contactPhone.
Steps	1. Anonymous user submits POST /api/requests/:id/resources with matching contactPhone in body 2. Verify requirement appended
Expected result	HTTP 201 Created with requirement saved based on phone match.
SRS requirement(s)	FR-23
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-RESC-10 should allow an admin and the assigned claimant to append or update requirements

Type, polarity, roles	Integration (Vitest and Supertest); Positive; admin, volunteer
Test location	server/tests/allocation.test.ts:649 Resource Allocation: Partial vs. Full Fulfillment > Resource Requirement Authorization on Requests (POST /api/requests/:id/resources) > should allow an admin and the assigned claimant to append or update requirements
Code under test	server/src/routes/requests.ts:935-1076 POST /api/requests/:id/resources
Use case	Field responders on scene assess actual damage and adjust required medical supply quantities.
Preconditions	Request in_progress claimed by volunteer.
Steps	1. Claimed volunteer appends trauma kit requirement 2. Admin appends emergency generator requirement
Expected result	HTTP 201 Created for both assigned claimant and administrator.
SRS requirement(s)	FR-23
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-RESC-11 should register a resource against an Organization with initial inventory

Type, polarity, roles	Integration (Vitest and Supertest); Positive; resource_provider
Test location	server/tests/resources.test.ts:203 Resource Registration Endpoints > should register a resource against an Organization with initial inventory
Code under test	server/src/routes/resources.ts:9-111 POST /api/resources server/src/models/Resource.ts:18-36 Resource server/src/models/ResourceInventory.ts:26-44 ResourceInventory
Use case	A regional medical depot registers clean drinking water crates into the operational catalog with initial stock.
Preconditions	Depot provider associated with verified Organization.
Steps	1. Submit POST /api/resources with name “Clean Drinking Water”, category “water”, and initial quantity 500 2. Verify Resource document and ResourceInventory record created with quantityAvailable=500
Expected result	HTTP 201 Created returning new Resource and linked ResourceInventory record.
SRS requirement(s)	FR-22
Phase, last run	Phase 2; Pass (2026-11-01)

TC-RESC-12 should reject registering a resource under an organization the user does not belong to

Type, polarity, roles	Integration (Vitest and Supertest); Negative; resource_provider
Test location	server/tests/resources.test.ts:246 Resource Registration Endpoints > should reject registering a resource under an organization the user does not belong to
Code under test	server/src/routes/resources.ts:9-111 POST /api/resources
Use case	Prevents unauthorized users from registering catalog items on behalf of other relief agencies.
Preconditions	User belongs to Org A; submits request for Org B.
Steps	1. Submit POST /api/resources referencing foreign organizationId 2. Assert authorization rejection
Expected result	HTTP 403 Forbidden with organization membership error.
SRS requirement(s)	FR-22
Phase, last run	Phase 2; Pass (2026-11-01)

TC-RESC-13 should register a resource against an individual provider without an organization

Type, polarity, roles	Integration (Vitest and Supertest); Positive; resource_provider
Test location	server/tests/resources.test.ts:276 Resource Registration Endpoints > should register a resource against an individual provider without an organization
Code under test	server/src/routes/resources.ts:9-111 POST /api/resources
Use case	An independent civilian equipment owner registers a personal 4x4 rescue boat for emergency deployment.
Preconditions	Authenticated resource_provider without organization membership.
Steps	1. Submit POST /api/resources with equipment details without organizationId 2. Verify inventory record linked directly to provider userId
Expected result	HTTP 201 Created with individual inventory record registered.
SRS requirement(s)	FR-22
Phase, last run	Phase 2; Pass (2026-11-01)

TC-RESC-14 should reject resource registration if required fields are missing

Type, polarity, roles	Integration (Vitest and Supertest); Negative; resource provider
Test location	server/tests/resources.test.ts:302 Resource Registration Endpoints > should reject resource registration if required fields are missing
Code under test	server/src/routes/resources.ts:9-111 POST /api/resources
Use case	Guarantees data integrity by rejecting incomplete catalog submissions lacking name or unit.
Preconditions	Authenticated provider.
Steps	1. Submit POST /api/resources missing required field “name” 2. Assert validation failure
Expected result	HTTP 400 Bad Request with missing fields error.
SRS requirement(s)	FR-22
Phase, last run	Phase 2; Pass (2026-11-01)

TC-RESC-15 should reserve inventory correctly and update available/reserved quantities

Type, polarity, roles	Integration (Vitest and Supertest); Positive; resource provider
Test location	server/tests/resources.test.ts:325 Resource Inventory & Reserve/Release Operations > should reserve inventory correctly and update available/reserved quantities
Code under test	server/src/routes/inventory.ts:61-116 POST /api/inventory/:id/reserve server/src/models/ResourceInventory.ts:26-44 ResourceInventory
Use case	A depot manager earmarks 20 blankets for dispatch, moving them from available to reserved stock.
Preconditions	Depot inventory document exists with available quantity >= 20.
Steps	1. Submit POST /api/inventory/:id/reserve with quantity=20 2. Query updated inventory record
Expected result	HTTP 200 OK; quantityAvailable decreases by 20; quantityReserved increases by 20.
SRS requirement(s)	FR-25
Phase, last run	Phase 2; Pass (2026-11-01)

TC-RESC-16 should reject reserving more quantity than available

Type, polarity, roles	Integration (Vitest and Supertest); Negative; resource provider
Test location	server/tests/resources.test.ts:379 Resource Inventory & Reserve/Release Operations > should reject reserving more quantity than available
Code under test	server/src/routes/inventory.ts:61-116 POST /api/inventory/:id/reserve
Use case	Prevents double-booking or inventory overdraft by disallowing reservations exceeding available stock.
Preconditions	Depot stock has available quantity 10.
Steps	1. Submit POST /api/inventory/:id/reserve with quantity=50 2. Assert overdraft rejection
Expected result	HTTP 400 Bad Request indicating insufficient stock available.
SRS requirement(s)	FR-25
Phase, last run	Phase 2; Pass (2026-11-01)

TC-RESC-17 should release reserved inventory correctly back to available

Type, polarity, roles	Integration (Vitest and Supertest); Positive; resource provider
Test location	server/tests/resources.test.ts:419 Resource Inventory & Reserve/Release Operations > should release reserved inventory correctly back to available
Code under test	server/src/routes/inventory.ts:118-169 POST /api/inventory/:id/release
Use case	A stood-down convoy returns unused rations back to available depot inventory.
Preconditions	Inventory record with quantityReserved >= 15.
Steps	1. Submit POST /api/inventory/:id/release with quantity=15 2. Query updated inventory record
Expected result	HTTP 200 OK; quantityReserved decreases by 15; quantityAvailable increases by 15.
SRS requirement(s)	FR-25
Phase, last run	Phase 2; Pass (2026-11-01)

TC-RESC-18 should reject reserve and release from unauthorized roles: volunteer and affected_person receive 403 Forbidden

Type, polarity, roles	Integration (Vitest and Supertest); Negative; volunteer, affected person
Test location	server/tests/resources.test.ts:466 Resource Inventory & Reserve/Release Operations > should reject reserve and release from unauthorized roles: volunteer and affected person receive 403 Forbidden
Code under test	server/src/routes/inventory.ts:61-116 POST /api/inventory/:id/reserve server/src/routes/inventory.ts:118-169 POST /api/inventory/:id/release
Use case	Protects depot inventory balances by barring field volunteers and citizens from modifying stock reserves.
Preconditions	Existing inventory record.
Steps	1. Volunteer and citizen send POST /api/inventory/:id/reserve 2. Volunteer and citizen send POST /api/inventory/:id/release
Expected result	HTTP 403 Forbidden for all unauthorized reserve/release operations.
SRS requirement(s)	FR-25, FR-4
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-RESC-19 should view inventory with filters and populates

Type, polarity, roles	Integration (Vitest and Supertest); Positive; anonymous
Test location	server/tests/resources.test.ts:532 Resource Inventory & Reserve/Release Operations > should view inventory with filters and populates
Code under test	server/src/routes/inventory.ts:9-59 GET /api/inventory
Use case	Dispatchers filter available resources across relief depots by category and location.
Preconditions	Multiple inventories seeded.
Steps	1. Send GET /api/inventory with filter query parameters 2. Verify populated resource and organization references
Expected result	HTTP 200 OK with filtered list and populated resource specifications.
SRS requirement(s)	FR-22
Phase, last run	Phase 2; Pass (2026-11-01)

3.5 Messaging and notifications

Provides request-scoped message exchange between verified responders and victims, telephone-authenticated bystander comms, and in-app lifecycle event notification dispatching and read-state management.

TC-COMM-01 should allow admin and requester to exchange messages on unverified and verified requests, while rejecting unrelated volunteers

Type, polarity, roles	Integration (Vitest and Supertest); Positive; admin, affected person, volunteer
Test location	server/tests/messages.test.ts:45 Request-Scoped Messaging & Thread Authorization > should allow admin and requester to exchange messages on unverified and verified requests, while rejecting unrelated volunteers
Code under test	server/src/routes/requests.ts:1365-1404 GET /api/requests/:id/messages server/src/routes/requests.ts:1405-1503 POST /api/requests/:id/messages server/src/models/Message.ts:17-38 Message
Use case	Enables triage coordination between dispatcher and victim before responder dispatch, while blocking unrelated volunteers.
Preconditions	Ticket in unverified status created by citizen.
Steps	1. Dispatcher and citizen exchange messages in thread via POST /api/requests/:id/messages 2. Unrelated volunteer attempts to read and post in thread
Expected result	Dispatcher and creator exchange messages successfully; unrelated volunteer rejected with 403 Forbidden.
SRS requirement(s)	FR-27
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-COMM-02 should allow only authorized participants (claimant, requester, admin) to post and read

Type, polarity, roles	Integration (Vitest and Supertest); Positive; volunteer, affected person, admin
Test location	server/tests/messages.test.ts:163 Request-Scoped Messaging & Thread Authorization > should allow only authorized participants (claimant, requester, admin) to post and read
Code under test	server/src/routes/requests.ts:1365-1404 GET /api/requests/:id/messages server/src/routes/requests.ts:1405-1503 POST /api/requests/:id/messages
Use case	Provides secure mission coordination channel restricted to assigned responder, victim, and central dispatch.
Preconditions	Request in_progress claimed by volunteer.
Steps	1. Volunteer and citizen exchange tactical notes in request channel 2. Verify messages stored with timestamps and sender role tags
Expected result	HTTP 201 Created on posts; messages retrieved in chronological order for authorized participants.
SRS requirement(s)	FR-27
Phase, last run	Phase 2; Pass (2026-11-01)

TC-COMM-03 should allow anonymous requesters to read and post via phone verification

Type, polarity, roles	Integration (Vitest and Supertest); Positive; anonymous
Test location	server/tests/messages.test.ts:278 Request-Scoped Messaging & Thread Authorization > should allow anonymous requesters to read and post via phone verification
Code under test	server/src/routes/requests.ts:1365-1404 GET /api/requests/:id/messages server/src/routes/requests.ts:1405-1503 POST /api/requests/:id/messages
Use case	Enables an anonymous civilian without an account to chat with their rescuer by verifying their phone number.
Preconditions	Anonymous request filed with contactPhone registered.
Steps	1. Anonymous user posts and reads messages passing contactPhone in request header/body 2. Verify phone matches intake record
Expected result	HTTP 201 Created and HTTP 200 OK permitting anonymous communication.
SRS requirement(s)	FR-27, FR-28
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-COMM-04 should generate notifications for requester on verify, claim, and resolve events

Type, polarity, roles	Integration (Vitest and Supertest); Positive; affected person, admin, volunteer
Test location	server/tests/notifications.test.ts:42 Automatic Notifications on Lifecycle Events > should generate notifications for requester on verify, claim, and resolve events
Code under test	server/src/routes/notifications.ts:9-40 GET /api/notifications server/src/models/Notification.ts:16-36 Notification
Use case	Notifies victim in-app as their rescue ticket transitions through verify, claim, and resolution milestones.
Preconditions	Ticket submitted by registered citizen.
Steps	1. Admin verifies ticket, volunteer claims ticket, volunteer resolves ticket 2. Query Notification collection for citizen recipient
Expected result	Three in-app notifications generated with descriptive event summaries and read=false.
SRS requirement(s)	FR-29
Phase, last run	Phase 2; Pass (2026-11-01)

TC-COMM-05 should not throw error or create orphan notifications for anonymous submissions

Type, polarity, roles	Integration (Vitest and Supertest); Positive; anonymous
Test location	server/tests/notifications.test.ts:134 Automatic Notifications on Lifecycle Events > should not throw error or create orphan notifications for anonymous submissions
Code under test	server/src/routes/requests.ts:272-646 Lifecycle notification dispatcher
Use case	Ensures lifecycle transitions on anonymous bystander tickets complete smoothly without generating orphan database notifications.
Preconditions	Anonymous intake ticket without createdBy user account.
Steps	1. Transition anonymous ticket through verify, claim, and resolve 2. Verify zero orphan notification records created
Expected result	Lifecycle actions return 200 OK cleanly; system skips notification creation without error.
SRS requirement(s)	FR-29
Phase, last run	Phase 2; Pass (2026-11-01)

TC-COMM-06 should fetch user notifications with unreadCount and support marking as read

Type, polarity, roles	Integration (Vitest and Supertest); Positive; volunteer
Test location	server/tests/notifications.test.ts:183 Notification API Endpoints > should fetch user notifications with unreadCount and support marking as read
Code under test	server/src/routes/notifications.ts:9-40 GET /api/notifications server/src/routes/notifications.ts:42-75 PATCH /api/notifications/:id/read server/src/routes/notifications.ts:77-101 PATCH /api/notifications/read-all
Use case	Responder checks unread notifications bell in console navigation and dismisses alerts once read.
Preconditions	User has unread notifications stored.
Steps	1. Send GET /api/notifications to retrieve list and unreadCount 2. Send PATCH /api/notifications/:id/read to mark single alert read 3. Send PATCH /api/notifications/read-all to clear unread badge
Expected result	HTTP 200 OK; unreadCount decrements; notifications marked read=true.
SRS requirement(s)	FR-29
Phase, last run	Phase 2; Pass (2026-11-01)

3.6 Evidence

Enables local disk upload and retrieval of visual, photographic, and document proof supporting emergency requests, including anonymous informant uploads and file validation.

TC-EVID-01 should correctly upload evidence and link metadata to request and uploader

Type, polarity, roles	Integration (Vitest and Supertest); Positive; volunteer
Test location	server/tests/evidence.test.ts:55 Evidence Attachment Metadata & Linking > should correctly upload evidence and link metadata to request and uploader
Code under test	server/src/routes/requests.ts:1504-1556 POST /api/requests/:id/evidence server/src/routes/requests.ts:1557-1586 GET /api/requests/:id/evidence server/src/models/Evidence.ts:17-38 Evidence
Use case	Field responder uploads on-scene damage photo to justify emergency boat deployment.
Preconditions	Authenticated user and existing request ticket.
Steps	1. Submit multipart/form-data POST /api/requests/:id/evidence with test image buffer 2. Query Evidence collection and GET /api/requests/:id/evidence
Expected result	HTTP 201 Created; file saved to local disk; Evidence record linked to ticket with MIME type and size.
SRS requirement(s)	FR-30
Phase, last run	Phase 2; Pass (2026-11-01)

TC-EVID-02 should allow anonymous informants to upload supporting evidence with fallback uploader name

Type, polarity, roles	Integration (Vitest and Supertest); Positive; anonymous
Test location	server/tests/evidence.test.ts:111 Evidence Attachment Metadata & Linking > should allow anonymous informants to upload supporting evidence with fallback uploader name
Code under test	server/src/routes/requests.ts:1504-1556 POST /api/requests/:id/evidence
Use case	A bystander uploads a photo of a collapsed bridge without logging in to assist emergency routing.
Preconditions	Public intake ticket active.
Steps	1. Anonymous user uploads photo via POST /api/requests/:id/evidence with uploaderName “Local Bystander” 2. Verify saved evidence document metadata
Expected result	HTTP 201 Created with uploader marked as anonymous bystander informant.
SRS requirement(s)	FR-30
Phase, last run	Phase 2; Pass (2026-11-01)

TC-EVID-03 should reject upload if no file is provided

Type, polarity, roles	Integration (Vitest and Supertest); Negative; volunteer
Test location	server/tests/evidence.test.ts:141 Evidence Attachment Metadata & Linking > should reject upload if no file is provided
Code under test	server/src/routes/requests.ts:1504-1556 POST /api/requests/:id/evidence
Use case	Guarantees multipart integrity by rejecting evidence submissions lacking an attachment file.
Preconditions	Authenticated user.
Steps	1. Submit POST /api/requests/:id/evidence without multipart file payload 2. Assert upload validation rejection
Expected result	HTTP 400 Bad Request with missing file error.
SRS requirement(s)	FR-30
Phase, last run	Phase 2; Pass (2026-11-01)

3.7 Categories and incidents

Manages the dynamic emergency category taxonomy and the creation of geographic major incidents that cluster multi-ticket disasters within bounding perimeters.

TC-CAT-01 should seed the 5 existing default categories and allow end-to-end request creation with them

Type, polarity, roles	Integration (Vitest and Supertest); Positive; anonymous
Test location	server/tests/categories_incidents.test.ts:46 EmergencyCategory Entity & Backward Compatibility > should seed the 5 existing default categories and allow end-to-end request creation with them
Code under test	server/src/routes/categories.ts:8-32 GET /api/categories server/src/models/EmergencyCategory.ts:15-55 EmergencyCategory
Use case	Guarantees the 5 canonical crisis categories (medical, rescue, supplies, shelter, other) are active and usable.
Preconditions	Default categories initialized in database.
Steps	1. Send GET /api/categories to confirm active categories 2. Submit emergency request with category “medical”
Expected result	HTTP 200 OK listing 5 active categories; request created successfully referencing category document.
SRS requirement(s)	FR-7, FR-34
Phase, last run	Phase 2; Pass (2026-11-01)

TC-CAT-02 should allow admin to create and update new categories, while forbidding non-admins

Type, polarity, roles	Integration (Vitest and Supertest); Positive; admin, volunteer
Test location	server/tests/categories_incidents.test.ts:78 EmergencyCategory Entity & Backward Compatibility > should allow admin to create and update new categories, while forbidding non-admins
Code under test	server/src/routes/categories.ts:33-72 POST /api/categories server/src/routes/categories.ts:74-108 PATCH /api/categories/:id
Use case	Allows dispatch directors to dynamically create new crisis categories (e.g. hazmat) while blocking non-admins.
Preconditions	Admin and volunteer authenticated.
Steps	1. Admin creates new category “hazmat” via POST /api/categories 2. Volunteer attempts to create category (blocked with 403) 3. Admin updates category priority level via PATCH /api/categories/:id
Expected result	Admin creates and updates category with 201/200; volunteer rejected with 403 Forbidden.
SRS requirement(s)	FR-34, FR-4
Phase, last run	Phase 2; Pass (2026-11-01)

TC-CAT-03 should allow admin to create an incident, link requests, and retrieve grouped requests

Type, polarity, roles	Integration (Vitest and Supertest); Positive; admin
Test location	server/tests/categories_incidents.test.ts:138 Incident Entity & Grouped Request Management > should allow admin to create an incident, link requests, and retrieve grouped requests
Code under test	server/src/routes/incidents.ts:23-89 POST /api/incidents server/src/routes/incidents.ts:139-206 POST /api/incidents/:id/requests server/src/routes/incidents.ts:208-238 GET /api/incidents/:id/requests server/src/models/Incident.ts:27-90 Incident
Use case	Clusters multiple flood rescue calls occurring in Sector 4 under a single major incident commander view.
Preconditions	Admin authenticated; multiple open requests exist.
Steps	1. Admin declares major incident via POST /api/incidents with perimeter coordinates and radius 2. Admin links related requests via POST /api/incidents/:id/requests 3. Retrieve grouped requests via GET /api/incidents/:id/requests
Expected result	Incident created with unique code; requests linked; grouped list returns requests under incident banner.
SRS requirement(s)	FR-33
Phase, last run	Phase 2; Pass (2026-11-01)

3.8 Organizations

Supports registration, directory listing, and profile administration of relief organizations (NGOs, government depots, and medical units) and collective org-level claim ownership.

TC-ORG-01 should support org-level claim ownership: allow teammate B to resolve request claimed by teammate A and participate in messages

Type, polarity, roles	Integration (Vitest and Supertest); Positive; ngo
Test location	server/tests/org_claim_and_ngo_verify.test.ts:233 Org-Level Claim Ownership & Verified NGO Verification Authority > should support org-level claim ownership: allow teammate B to resolve request claimed by teammate A and participate in messages
Code under test	server/src/routes/requests.ts:454-546 PATCH /api/requests/:id/claim server/src/routes/requests.ts:547-646 PATCH /api/requests/:id/resolve
Use case	Enables relief team shift handovers: NGO Responder B resolves a mission originally claimed by NGO Responder A.
Preconditions	Responder A and B belong to same verified NGO; Request claimed by Responder A with claimedOrg set.
Steps	1. Responder A claims verified ticket 2. Responder B posts in request message thread 3. Responder B marks request resolved via PATCH /api/requests/:id/resolve
Expected result	Responder B permitted to message and resolve based on shared organization ownership.
SRS requirement(s)	FR-13, FR-27
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-ORG-02 should create an organization and link it to user when user has no organization

Type, polarity, roles	Integration (Vitest and Supertest); Positive; ngo
Test location	server/tests/resources.test.ts:45 Organization Management Endpoints > should create an organization and link it to user when user has no organization
Code under test	server/src/routes/organizations.ts:9-110 POST /api/organizations server/src/models/Organization.ts:24-42 Organization
Use case	An NGO lead registers their relief charity on the platform, linking their account as primary contact.
Preconditions	User authenticated with NGO role without existing organization link.
Steps	1. Submit POST /api/organizations with name, type “ngo”, email, phone, and address 2. Verify organization document created and user.organizationId updated
Expected result	HTTP 201 Created with organization details; user profile linked.
SRS requirement(s)	FR-35
Phase, last run	Phase 2; Pass (2026-11-01)

TC-ORG-03 should reject creating an organization with duplicate name

Type, polarity, roles	Integration (Vitest and Supertest); Negative; ngo
Test location	server/tests/resources.test.ts:80 Organization Management Endpoints > should reject creating an organization with duplicate name
Code under test	server/src/routes/organizations.ts:9-110 POST /api/organizations
Use case	Prevents fraudulent or duplicate organization profiles claiming identical agency names.
Preconditions	Organization with target name already registered.
Steps	1. Submit POST /api/organizations with duplicate organization name 2. Assert conflict rejection
Expected result	HTTP 409 Conflict with duplicate organization name error.
SRS requirement(s)	FR-35
Phase, last run	Phase 2; Pass (2026-11-01)

TC-ORG-04 should reject creating an organization with invalid type

Type, polarity, roles	Integration (Vitest and Supertest); Negative; ngo
Test location	server/tests/resources.test.ts:106 Organization Management Endpoints > should reject creating an organization with invalid type
Code under test	server/src/routes/organizations.ts:9-110 POST /api/organizations
Use case	Validates organizational taxonomy (must be ngo, government, medical, or corporate_donor).
Preconditions	Authenticated user.
Steps	1. Submit POST /api/organizations with unrecognized type "commercial_shop" 2. Assert validation failure
Expected result	HTTP 400 Bad Request with valid organization types enumeration error.
SRS requirement(s)	FR-35
Phase, last run	Phase 2; Pass (2026-11-01)

TC-ORG-05 should list and retrieve organizations

Type, polarity, roles	Integration (Vitest and Supertest); Positive; anonymous
Test location	server/tests/resources.test.ts:127 Organization Management Endpoints > should list and retrieve organizations
Code under test	server/src/routes/organizations.ts:116-140 GET /api/organizations server/src/routes/organizations.ts:142-166 GET /api/organizations/:id
Use case	Allows responders and public to view registered crisis relief organizations in the platform directory.
Preconditions	Multiple organizations seeded in database.
Steps	1. Send GET /api/organizations 2. Send GET /api/organizations/:id for specific organization profile
Expected result	HTTP 200 OK returning array of active relief organizations and populated primary contacts.
SRS requirement(s)	FR-35
Phase, last run	Phase 2; Pass (2026-11-01)

TC-ORG-06 should allow primary contact to update their own organization via POST /api/organizations

Type, polarity, roles	Integration (Vitest and Supertest); Positive; ngo
Test location	server/tests/resources.test.ts:157 Organization Management Endpoints > should allow primary contact to update their own organization via POST /api/organizations
Code under test	server/src/routes/organizations.ts:9-110 POST /api/organizations
Use case	Primary contact updates agency contact details, operating address, or mission profile.
Preconditions	User is primaryContact of target Organization.
Steps	1. Submit POST /api/organizations with updated phone and operating sector 2. Verify updated fields in database
Expected result	HTTP 200 OK with updated profile information.
SRS requirement(s)	FR-35
Phase, last run	Phase 2B; Pass (2026-11-01)

3.9 Verification

Audits the intake assessment step, recording verifier identity, remarks, assessment method, and priority updates, while enforcing anti-self-verification conflict-of-interest protections.

TC-VRFY-01 should allow a verified NGO member to verify an unverified request, while rejecting unverified NGOs and volunteers

Type, polarity, roles	Integration (Vitest and Supertest); Positive; ngo, volunteer
Test location	server/tests/org_claim_and_ngo_verify.test.ts:54 Org-Level Claim Ownership & Verified NGO Verification Authority > should allow a verified NGO member to verify an unverified request, while rejecting unverified NGOs and volunteers
Code under test	server/src/routes/requests.ts:272-422 PATCH /api/requests/:id/verify server/src/models/Organization.ts:24-42 Organization
Use case	Permits trusted relief NGOs with verified organizations to verify intake tickets while blocking unverified organizations.
Preconditions	Organization verified=true and user associated with org.
Steps	1. Verified NGO member sends PATCH /api/requests/:id/verify 2. Unverified NGO and volunteer send same request
Expected result	Verified NGO succeeds with 200 OK; unverified NGO and volunteer rejected with 403 Forbidden.
SRS requirement(s)	FR-10
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-VRFY-02 should reject verified NGO from verifying their own ticket or tickets created by their teammates (Conflict of Interest)

Type, polarity, roles	Integration (Vitest and Supertest); Negative; ngo
Test location	server/tests/org_claim_and_ngo_verify.test.ts:164 Org-Level Claim Ownership & Verified NGO Verification Authority > should reject verified NGO from verifying their own ticket or tickets created by their teammates (Conflict of Interest)
Code under test	server/src/routes/requests.ts:272-422 PATCH /api/requests/:id/verify
Use case	Prevents organizational bias by blocking NGO coordinators from verifying requests reported by their own team.
Preconditions	Ticket reported by Member A of Relief NGO.
Steps	1. Member A attempts to verify ticket 2. Member B (same NGO) attempts to verify ticket
Expected result	Both rejected with 403 Forbidden due to organizational conflict of interest.
SRS requirement(s)	FR-11
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-VRFY-03 should create both a StatusHistory entry and a Verification record when remarks are provided

Type, polarity, roles	Integration (Vitest and Supertest); Positive; admin
Test location	server/tests/verification.test.ts:42 Verification Entity Integration > should create both a StatusHistory entry and a Verification record when remarks are provided
Code under test	server/src/routes/requests.ts:272-422 PATCH /api/requests/:id/verify server/src/models/Verification.ts:18-42 Verification
Use case	Dispatcher verifies a phone report and logs intake remarks, assessment method, and priority updates.
Preconditions	Unverified ticket in database.
Steps	1. Admin sends PATCH /api/requests/:id/verify with remarks, method “phone call”, and priority “high” 2. Query Verification collection and Request.statusHistory
Expected result	Ticket verified; Verification record created linked to request; StatusHistory entry appended.
SRS requirement(s)	FR-10, FR-16
Phase, last run	Phase 2; Pass (2026-11-01)

TC-VRFY-04 should verify cleanly and not break anything when remarks are completely omitted

Type, polarity, roles	Integration (Vitest and Supertest); Positive; admin
Test location	server/tests/verification.test.ts:102 Verification Entity Integration > should verify cleanly and not break anything when remarks are completely omitted
Code under test	server/src/routes/requests.ts:272-422 PATCH /api/requests/:id/verify
Use case	Guarantees backwards compatibility by allowing dispatchers to verify requests without entering optional remarks or notes.
Preconditions	Unverified ticket submitted; admin user logged in.
Steps	1. Admin sends PATCH /api/requests/:id/verify with empty request body 2. Verify status transitions cleanly to verified 3. Verify StatusHistory fallback note is populated and Verification record exists with undefined remarks
Expected result	HTTP 200 OK with status: verified and default verification values.
SRS requirement(s)	FR-10
Phase, last run	Phase 2; Pass (2026-11-01)

TC-VRFY-05 should transition request to closed when marked duplicate or unfounded

Type, polarity, roles	Integration (Vitest and Supertest); Positive; admin
Test location	server/tests/verification.test.ts:145 Verification Entity Integration > should transition request to closed when marked duplicate or unfounded
Code under test	server/src/routes/requests.ts:272-422 PATCH /api/requests/:id/verify
Use case	Allows dispatchers assessing incoming calls to instantly close tickets that are verified as duplicate or unfounded reports.
Preconditions	Unverified ticket submitted.
Steps	1. Admin sends PATCH /api/requests/:id/verify with result: “unfounded” 2. Verify response status is 200 and request status transitions immediately to closed 3. Confirm closed entry in StatusHistory
Expected result	HTTP 200 OK with request status transitioning directly to closed.
SRS requirement(s)	FR-10, FR-14
Phase, last run	Phase 2; Pass (2026-11-01)

TC-VRFY-06 should reject an admin attempting to verify their own submitted request, while allowing verification of other requests

Type, polarity, roles	Integration (Vitest and Supertest); Negative; admin
Test location	server/tests/verification.test.ts:194 Verification Entity Integration > should reject an admin attempting to verify their own submitted request, while allowing verification of other requests
Code under test	server/src/routes/requests.ts:272-422 PATCH /api/requests/:id/verify
Use case	Prevents conflict of interest by disallowing dispatchers from verifying emergency tickets they created themselves.
Preconditions	Admin A submitted Request A; Admin B submitted Request B.
Steps	1. Admin A attempts to verify Request A (self-verification) 2. Admin B attempts to verify Request A (peer verification)
Expected result	Admin A rejected with 403 Forbidden (Conflict of interest); Admin B permitted with 200 OK.
SRS requirement(s)	FR-11
Phase, last run	Phase 3B; Pass (2026-11-01)

3.10 Cancellation

Implements the cancellation and withdrawal protocol, allowing victims and dispatchers to withdraw tickets while guaranteeing automated inventory release and assignment stand-down.

TC-CNCL-01 should allow the authenticated creator to withdraw an unverified or verified request

Type, polarity, roles	Integration (Vitest and Supertest); Positive; affected person
Test location	server/tests/cancellation.test.ts:54 Request Cancellation & Withdrawal Lifecycle (Option A) > should allow the authenticated creator to withdraw an unverified or verified request
Code under test	server/src/routes/requests.ts:725-934 PATCH /api/requests/:id/cancel
Use case	A displaced citizen who relocated to safety withdraws their pending call for help.
Preconditions	Request created by authenticated citizen in state unverified or verified.
Steps	1. Send PATCH /api/requests/:id/cancel with reason “Situation resolved by neighbors” 2. Verify status transitions to cancelled and statusHistory records withdrawal
Expected result	HTTP 200 OK with status “cancelled” and reason logged in statusHistory.
SRS requirement(s)	FR-15
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-CNCL-02 should allow withdrawal during in_progress, releasing reserved stock and standing down assignments

Type, polarity, roles	Integration (Vitest and Supertest); Positive; affected_person
Test location	server/tests/cancellation.test.ts:104 Request Cancellation & Withdrawal Lifecycle (Option A) > should allow withdrawal during in_progress, releasing reserved stock and standing down assignments
Code under test	server/src/routes/requests.ts:725-934 PATCH /api/requests/:id/cancel server/src/models/ResourceInventory.ts:26-44 ResourceInventory server/src/models/RequestAssignment.ts:22-38 RequestAssignment
Use case	Withdrawing an in-progress ticket automatically releases reserved relief supplies and stands down assigned units.
Preconditions	Request in_progress with active RequestAssignment and reserved inventory.
Steps	1. Creator cancels active in_progress request 2. Verify RequestAssignment status updated to cancelled 3. Verify ResourceInventory reserved quantity returned to available stock
Expected result	HTTP 200 OK; assignment marked cancelled; reserved inventory decremented and available restored.
SRS requirement(s)	FR-15, FR-25
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-CNCL-03 should allow anonymous requesters to withdraw via contact phone matching

Type, polarity, roles	Integration (Vitest and Supertest); Positive; anonymous
Test location	server/tests/cancellation.test.ts:195 Request Cancellation & Withdrawal Lifecycle (Option A) > should allow anonymous requesters to withdraw via contact phone matching
Code under test	server/src/routes/requests.ts:725-934 PATCH /api/requests/:id/cancel
Use case	Bystander who submitted an emergency ticket without creating an account cancels via verified contact phone.
Preconditions	Ticket submitted anonymously with contact phone.
Steps	1. Send PATCH /api/requests/:id/cancel with matching contactPhone and reason 2. Verify phone match grants cancellation authorization
Expected result	HTTP 200 OK with status updated to cancelled.
SRS requirement(s)	FR-15
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-CNCL-04 should allow administrators to cancel at any point before closed, while blocking unrelated volunteers

Type, polarity, roles	Integration (Vitest and Supertest); Positive; admin, volunteer
Test location	server/tests/cancellation.test.ts:231 Request Cancellation & Withdrawal Lifecycle (Option A) > should allow administrators to cancel at any point before closed, while blocking unrelated volunteers
Code under test	server/src/routes/requests.ts:725-934 PATCH /api/requests/:id/cancel
Use case	Dispatchers can cancel false alarms or duplicates, while unrelated field volunteers are strictly blocked.
Preconditions	Active request ticket.
Steps	1. Unrelated volunteer attempts cancellation (blocked with 403) 2. Admin submits cancellation with reason “Duplicate report” (permitted)
Expected result	Volunteer rejected with 403 Forbidden; admin permitted with 200 OK.
SRS requirement(s)	FR-15, FR-4
Phase, last run	Phase 3B; Pass (2026-11-01)

TC-CNCL-05 should confirm existing resolve and confirm flow for non-cancelled requests is unaffected

Type, polarity, roles	Integration (Vitest and Supertest); Positive; volunteer, affected person
Test location	server/tests/cancellation.test.ts:285 Request Cancellation & Withdrawal Lifecycle (Option A) > should confirm existing resolve and confirm flow for non-cancelled requests is unaffected
Code under test	server/src/routes/requests.ts:547-646 PATCH /api/requests/:id/resolve server/src/routes/requests.ts:647-724 PATCH /api/requests/:id/confirm
Use case	Verifies that adding the cancellation flow introduces zero regressions to regular resolution and confirmation.
Preconditions	Regular active ticket in progress.
Steps	1. Volunteer resolves claimed request via PATCH /api/requests/:id/resolve 2. Citizen confirms resolution via PATCH /api/requests/:id/confirm
Expected result	Ticket moves from in progress to resolved to closed cleanly.
SRS requirement(s)	FR-13, FR-14
Phase, last run	Phase 3B; Pass (2026-11-01)

3.11 Real-time (Socket.IO)

Handles WebSocket connection handshakes, room subscriptions, client authentication, and post-DB-write real-time event broadcasting (request:created, request:updated, allocation:updated, message:new, notification:new) with dynamic privacy sanitization.

TC-SOCK-01 allows an authenticated user to connect with valid JWT and joins public room

Type, polarity, roles	Integration (Vitest and Supertest); Positive; volunteer
Test location	server/tests/socket.test.ts:53 Socket.IO Real-Time Engine & Handshake Authentication > allows an authenticated user to connect with valid JWT and joins public room
Code under test	server/src/socket/index.ts:81-315 initSocketIO
Use case	Field volunteer establishes live WebSocket connection to receive real-time incident broadcasts.
Preconditions	Server Socket.IO engine attached; valid user JWT.
Steps	1. Connect Socket.IO client with auth: *{* token: <jwt> *}* 2. Verify connection accepted and socket joined public and personal user rooms
Expected result	WebSocket handshake succeeds; client receives connection ack.
SRS requirement(s)	FR-20
Phase, last run	Phase 3; Pass (2026-11-01)

TC-SOCK-02 allows an anonymous bystander to connect without token and joins public room

Type, polarity, roles	Integration (Vitest and Supertest); Positive; anonymous
Test location	server/tests/socket.test.ts:81 Socket.IO Real-Time Engine & Handshake Authentication > allows an anonymous bystander to connect without token and joins public room
Code under test	server/src/socket/index.ts:81-315 initSocketIO
Use case	Allows anonymous public citizen visiting the tracking page to receive live fuzzed emergency updates.
Preconditions	Server Socket.IO active.
Steps	1. Connect Socket.IO client without auth token 2. Verify connection permitted and client joined to public broadcast room
Expected result	Anonymous connection established successfully in public room.
SRS requirement(s)	FR-20, FR-1
Phase, last run	Phase 3; Pass (2026-11-01)

TC-SOCK-03 rejects connection when an invalid or corrupt JWT is provided in handshake

Type, polarity, roles	Integration (Vitest and Supertest); Negative; anonymous
Test location	server/tests/socket.test.ts:108 Socket.IO Real-Time Engine & Handshake Authentication > rejects connection when an invalid or corrupt JWT is provided in handshake
Code under test	server/src/socket/index.ts:81-315 initSocketIO
Use case	Prevents corrupted or tampered credentials from establishing an authenticated real-time channel.
Preconditions	Forged JWT string.
Steps	1. Attempt Socket.IO handshake with corrupt JWT token 2. Listen for connect_error event
Expected result	Handshake rejected with Authentication error: Invalid or expired token.
SRS requirement(s)	FR-20
Phase, last run	Phase 3; Pass (2026-11-01)

TC-SOCK-04 allows authenticated clients to subscribe to per-request channels and receive scoped updates

Type, polarity, roles	Integration (Vitest and Supertest); Positive; volunteer
Test location	server/tests/socket.test.ts:128 Socket.IO Real-Time Engine & Handshake Authentication > allows authenticated clients to subscribe to per-request channels and receive scoped updates
Code under test	server/src/socket/index.ts:81-315 initSocketIO
Use case	Assigned responder subscribes to specific incident channel to receive scoped dispatch instructions.
Preconditions	Authenticated socket connection established.
Steps	1. Emit request:subscribe with target requestId 2. Emit request:unsubscribe to detach
Expected result	Socket joins request-scoped room and leaves cleanly upon unsubscription.
SRS requirement(s)	FR-20
Phase, last run	Phase 3; Pass (2026-11-01)

TC-SOCK-05 denies anonymous clients from joining request-specific channels

Type, polarity, roles	Integration (Vitest and Supertest); Negative; anonymous
Test location	server/tests/socket.test.ts:169 Socket.IO Real-Time Engine & Handshake Authentication > denies anonymous clients from joining request-specific channels
Code under test	server/src/socket/index.ts:81-315 initSocketIO
Use case	Guarantees operational isolation by disallowing unauthenticated bystanders from eavesdropping on private channels.
Preconditions	Anonymous socket connection in public room.
Steps	1. Anonymous client emits request:subscribe for a private request channel 2. Assert server channel authorization rejection
Expected result	Subscription denied with unauthorized error; socket barred from joining room.
SRS requirement(s)	FR-20
Phase, last run	Phase 3; Pass (2026-11-01)

TC-SOCK-06 broadcasts request:created with sanitized coordinates to public room on request submission

Type, polarity, roles	Integration (Vitest and Supertest); Positive; anonymous
Test location	server/tests/socket_broadcast.test.ts:91 Real-Time Event Broadcasting with Privacy Redaction > broadcasts request:created with sanitized coordinates to public room on request submission
Code under test	server/src/socket/index.ts:358-371 broadcastRequestCreated server/src/utils/sanitize.ts:6-24 sanitizeRequest
Use case	New emergency call broadcast live to all active map monitors with exact coordinates safely fuzzed.
Preconditions	Socket clients listening on public room.
Steps	1. Submit new emergency request via POST /api/requests 2. Assert request:created event received by listening client
Expected result	Event payload contains fuzzed publicLat/publicLng; exact coordinates and address omitted.
SRS requirement(s)	FR-19, FR-20
Phase, last run	Phase 3; Pass (2026-11-01)

TC-SOCK-07 emits request:updated with privacy redaction when a request is claimed

Type, polarity, roles	Integration (Vitest and Supertest); Positive; volunteer
Test location	server/tests/socket_broadcast.test.ts:134 Real-Time Event Broadcasting with Privacy Redaction > emits request:updated with privacy redaction when a request is claimed
Code under test	server/src/socket/index.ts:373-430 broadcastRequestUpdated
Use case	When a volunteer claims a ticket, live operations dashboards update the status chip to in_progress instantly.
Preconditions	Public client connected.
Steps	1. Volunteer claims verified request 2. Assert request:updated event received in public channel
Expected result	Public client receives sanitized update with status in progress and responder assignment.
SRS requirement(s)	FR-19, FR-20
Phase, last run	Phase 3; Pass (2026-11-01)

TC-SOCK-08 does NOT emit any socket event if a database write fails

Type, polarity, roles	Integration (Vitest and Supertest); Negative; anonymous
Test location	server/tests/socket_broadcast.test.ts:230 Real-Time Event Broadcasting with Privacy Redaction > does NOT emit any socket event if a database write fails
Code under test	server/src/routes/requests.ts:104-180 POST /api/requests server/src/socket/index.ts:358-371 broadcastRequestCreated
Use case	Enforces source-of-truth integrity: no ghost socket events are ever broadcast if MongoDB write fails.
Preconditions	Database transaction or validation will fail.
Steps	1. Attempt request creation with invalid category triggering database rejection 2. Verify zero socket events emitted
Expected result	Database error returned; no socket event emitted to any room.
SRS requirement(s)	FR-20
Phase, last run	Phase 3; Pass (2026-11-01)

TC-SOCK-09 broadcasts allocation:updated to request room and notification:new to recipient when a provider allocates resources

Type, polarity, roles	Integration (Vitest and Supertest); Positive; resource_provider
Test location	server/tests/socket_broadcast.test.ts:259 Real-Time Event Broadcasting with Privacy Redaction > broadcasts allocation:updated to request room and notification:new to recipient when a provider allocates resources
Code under test	server/src/socket/index.ts:432-448 broadcastAllocationUpdated server/src/socket/index.ts:450-462 broadcastNotificationNew
Use case	Notifies field responder instantly when a depot allocates water or blankets to their mission.
Preconditions	Volunteer subscribed to request channel.
Steps	1. Depot provider allocates resources to request 2. Assert allocation:updated event on request room and notification:new on volunteer personal room
Expected result	Both real-time events delivered to appropriate scoped rooms immediately after DB commit.
SRS requirement(s)	FR-20
Phase, last run	Phase 3; Pass (2026-11-01)

TC-SOCK-10 broadcasts message:new to authorized participants and rejects unauthorized volunteer

Type, polarity, roles	Integration (Vitest and Supertest); Positive; volunteer, affected person
Test location	server/tests/socket_broadcast.test.ts:378 Real-Time Event Broadcasting with Privacy Redaction > broadcasts message:new to authorized participants and rejects unauthorized volunteer
Code under test	server/src/socket/index.ts:464-490 broadcastMessageNew
Use case	Delivers real-time chat messages between rescuer and victim while isolating unauthorized observers.
Preconditions	Assigned volunteer and citizen connected to request room; unauthorized volunteer barred.
Steps	1. Volunteer posts message in thread 2. Verify message:new received by authorized participants and not by unrelated volunteer
Expected result	Real-time message delivered to authorized participants with sender role badge.
SRS requirement(s)	FR-20, FR-27
Phase, last run	Phase 3; Pass (2026-11-01)

3.12 Matching

Calculates multi-factor candidate scores (Haversine proximity, inventory suitability, capacity, and responder availability) for dispatch recommendations without automated assignment.

TC-MTCH-01 should calculate accurate Haversine distance in kilometers

Type, polarity, roles	Unit (Vitest); Positive; automated / internal
Test location	server/tests/matching.test.ts:55 Phase 4: Rule-Based Matching Engine Unit & Scoring Tests > should calculate accurate Haversine distance in kilometers
Code under test	server/src/services/matching.ts:52-75 calculateHaversineDistanceKm
Use case	Calculates accurate great-circle surface distance between responder coordinates and disaster site coordinates using Haversine formula.
Preconditions	Matching service loaded in memory.
Steps	1. Invoke calculateHaversineDistanceKm between known GPS coordinates (e.g., Patiala to Ludhiana) 2. Assert distance in kilometers matches spherical trigonometry reference within 0.1km margin
Expected result	Calculated distance matches known geographic ground-truth distance.
SRS requirement(s)	FR-31
Phase, last run	Phase 4; Pass (2026-11-01)

TC-MTCH-02 should score proximity higher for closer candidates and treat undeclared location as neutral (0.5)

Type, polarity, roles	Unit (Vitest); Positive; automated / internal
Test location	server/tests/matching.test.ts:63 Phase 4: Rule-Based Matching Engine Unit & Scoring Tests > should score proximity higher for closer candidates and treat undeclared location as neutral (0.5)
Code under test	server/src/services/matching.ts:87-240 calculateMatchesForRequest (Proximity Calculation)
Use case	Ranks candidates located within 5km higher than distant responders, while granting neutral 0.5 proximity score to responders with undeclared location.
Preconditions	Volunteers seeded with differing coordinates and undeclared coordinates.
Steps	1. Call calculateMatchesForRequest with request at target coordinate 2. Compare proximity score of local responder (1.2km away) vs distant responder (45km away) 3. Confirm undeclared responder receives exactly 0.5 neutral score without penalty
Expected result	Closest responder achieves proximity near 1.0; distant responder near 0.0; undeclared candidate receives 0.5.
SRS requirement(s)	FR-31
Phase, last run	Phase 4; Pass (2026-11-01)

TC-MTCH-03 should score candidates with matching inventory resources higher on suitability and capacity

Type, polarity, roles	Unit (Vitest); Positive; automated / internal
Test location	server/tests/matching.test.ts:139 Phase 4: Rule-Based Matching Engine Unit & Scoring Tests > should score candidates with matching inventory resources higher on suitability and capacity
Code under test	server/src/services/matching.ts:87-240 calculateMatchesForRequest (Suitability & Capacity Calculation)
Use case	Boosts suitability and capacity scores for resource providers and organizations whose depot inventory holds the exact requested supplies.
Preconditions	Request with requested items (e.g. 10 water packs); providers with 0 vs 20 water packs.
Steps	1. Invoke calculateMatchesForRequest against the request 2. Inspect suitability and capacity factor breakdown for provider holding 20 water packs 3. Compare against provider holding 0 water packs
Expected result	Equipped provider achieves 1.0 suitability and 1.0 capacity, ranking at the top.
SRS requirement(s)	FR-31
Phase, last run	Phase 4; Pass (2026-11-01)

TC-MTCH-04 should score undeclared availability neutrally (0.5) without penalizing candidate

Type, polarity, roles	Unit (Vitest); Positive; automated / internal
Test location	server/tests/matching.test.ts:214 Phase 4: Rule-Based Matching Engine Unit & Scoring Tests > should score undeclared availability neutrally (0.5) without penalizing candidate
Code under test	server/src/services/matching.ts:87-240 calculateMatchesForRequest (Availability Neutral Handling)
Use case	Prevents active volunteers from being unfairly disqualified from match recommendations when their availability flag is unconfigured.
Preconditions	Volunteer with undefined availability status seeded.
Steps	1. Invoke calculateMatchesForRequest for verified request 2. Assert availability factor for candidate is 0.5 3. Verify candidate overall score is computed properly without NaN or rejection
Expected result	Candidate receives neutral 0.5 availability factor and valid overall match score.
SRS requirement(s)	FR-31
Phase, last run	Phase 4; Pass (2026-11-01)

TC-MTCH-05 GET /api/requests/:id/matches: blocks unauthorized roles and returns ranked matches for admins/dispatchers

Type, polarity, roles	Integration (Vitest and Supertest); Positive; admin, volunteer, affected person
Test location	server/tests/matching.test.ts:256 Phase 4: Matching API Endpoints & Zero Auto-Assignment > GET /api/requests/:id/matches: blocks unauthorized roles and returns ranked matches for admins/dispatchers
Code under test	server/src/routes/requests.ts:1557-1585 GET /api/requests/:id/matches
Use case	Permits administrators to query ranked candidates for an emergency ticket while strictly blocking unauthorized volunteers or citizens.
Preconditions	Verified request and responder candidates exist in database.
Steps	1. Send GET /api/requests/:id/matches as affected person and verify 403 Forbidden 2. Send GET /api/requests/:id/matches as volunteer and verify 403 Forbidden 3. Send GET /api/requests/:id/matches as admin and verify 200 OK with sorted candidates array
Expected result	HTTP 403 for unauthorized roles; HTTP 200 with ranked candidate list and factor breakdowns for admin.
SRS requirement(s)	FR-31, FR-4
Phase, last run	Phase 4; Pass (2026-11-01)

TC-MTCH-06 POST /api/requests/:id/invite: dispatches an invitation notification without auto-claiming or changing ticket status

Type, polarity, roles	Integration (Vitest and Supertest); Positive; admin, volunteer
Test location	server/tests/matching.test.ts:330 Phase 4: Matching API Endpoints & Zero Auto-Assignment > POST /api/requests/:id/invite: dispatches an invitation notification without auto-claiming or changing ticket status
Code under test	server/src/routes/requests.ts:1587-1640 POST /api/requests/:id/invite
Use case	Dispatches an in-app match invitation notification to a responder without unilaterally assigning them or changing the ticket out of verified state.
Preconditions	Verified request and responder candidate seeded.
Steps	1. Send POST /api/requests/:id/invite with candidateId as admin 2. Assert response status 200 OK with invited candidate details 3. Check request in database and confirm status remains verified (not claimed) 4. Verify notification document created for candidate with type “match_invite”
Expected result	HTTP 200 OK; notification created for responder; zero auto-assignment occurs.
SRS requirement(s)	FR-32
Phase, last run	Phase 4; Pass (2026-11-01)

3.13 Analytics

Computes the core proposal evaluation metrics: median Time-to-Match, Double-Booking Rate (0% target), Requester Confirmation Rate, and WebSocket propagation latency telemetry.

TC-ANLT-01 Metric 1: Time-to-Match should accurately compute median and mean minutes between verification and assignment

Type, polarity, roles	Integration (Vitest and Supertest); Positive; automated / admin
Test location	server/tests/analytics.test.ts:53 Phase 4: Analytics Dashboard Proposal Metrics Service > Metric 1: Time-to-Match should accurately compute median and mean minutes between verification and assignment
Code under test	server/src/services/analytics.ts:52-289 getAnalyticsSummary
Use case	Calculates the project core evaluation metric: median and mean minutes elapsed between ticket verification and responder assignment.
Preconditions	Requests with known timestamps in StatusHistory for verified and in progress seeded.
Steps	1. Seed request 1 with 10-minute interval, request 2 with 20-minute interval, request 3 with 30-minute interval 2. Invoke getAnalyticsSummary for 7d range 3. Assert timeToMatch.medianMinutes equals 20.0 and meanMinutes equals 20.0
Expected result	Accurate median and mean minutes calculated from verified-to-claimed timestamps.
SRS requirement(s)	FR-36
Phase, last run	Phase 4; Pass (2026-11-01)

TC-ANLT-02 Metric 2: Double-Booking Rate should confirm 0% on clean state and detect concurrent conflicts when simulated

Type, polarity, roles	Integration (Vitest and Supertest); Positive; automated / admin
Test location	server/tests/analytics.test.ts:133 Phase 4: Analytics Dashboard Proposal Metrics Service > Metric 2: Double-Booking Rate should confirm 0% on clean state and detect concurrent conflicts when simulated
Code under test	server/src/services/analytics.ts:52-289 getAnalyticsSummary
Use case	Evaluates Proposal Metric 2 by validating that zero conflicting concurrent assignments exist (0% double-booking target).
Preconditions	Clean state requests with single active assignments seeded; simulated conflict injected.
Steps	1. Compute metrics on clean database and assert doubleBookingRate is 0.0% 2. Inject simulated duplicate active assignment on single ticket 3. Recompute metrics and verify double-booking conflict is detected with non-zero percentage
Expected result	0.0% double-booking rate on clean state; conflict properly detected upon simulation.
SRS requirement(s)	FR-36
Phase, last run	Phase 4; Pass (2026-11-01)

TC-ANLT-03 Metric 3: Confirmation Rate should compute percentage of resolved requests that reached closed via requester confirmation

Type, polarity, roles	Integration (Vitest and Supertest); Positive; automated / admin
Test location	server/tests/analytics.test.ts:206 Phase 4: Analytics Dashboard Proposal Metrics Service > Metric 3: Confirmation Rate should compute percentage of resolved requests that reached closed via requester confirmation
Code under test	server/src/services/analytics.ts:52-289 getAnalyticsSummary
Use case	Measures the fraction of resolved disaster requests that were confirmed as successfully satisfied by the affected citizen.
Preconditions	Requests in resolved status and closed status seeded.
Steps	1. Seed 2 resolved tickets and 2 closed tickets confirmed by requester 2. Invoke getAnalyticsSummary 3. Assert confirmationRate equals 50.0%
Expected result	Confirmation rate correctly computed as percentage of resolved tickets reaching closed.
SRS requirement(s)	FR-36
Phase, last run	Phase 4; Pass (2026-11-01)

TC-ANLT-04 Metric 4: Propagation Latency telemetry should record samples and compute median and p95 metrics

Type, polarity, roles	Unit (Vitest); Positive; automated / internal
Test location	server/tests/analytics.test.ts:239 Phase 4: Analytics Dashboard Proposal Metrics Service > Metric 4: Propagation Latency telemetry should record samples and compute median and p95 metrics
Code under test	server/src/socket/index.ts:30-50 recordTelemetrySample
Use case	Records WebSocket broadcast delivery latency samples and calculates real-time median and 95th-percentile propagation times.
Preconditions	Latency telemetry buffer reset.
Steps	1. Record series of latency duration samples (e.g. 10ms, 15ms, 20ms, 25ms, 100ms) 2. Invoke getLatencySummary 3. Assert median and p95 calculations match statistical expectations
Expected result	Accurate median and p95 latency telemetry returned.
SRS requirement(s)	FR-36
Phase, last run	Phase 4; Pass (2026-11-01)

TC-ANLT-05 GET /api/analytics should enforce strict role restrictions (admin only)

Type, polarity, roles	Integration (Vitest and Supertest); Positive; admin, volunteer, affected person, resource_provider
Test location	server/tests/analytics.test.ts:254 Phase 4: Analytics API Authorization & Security > GET /api/analytics should enforce strict role restrictions (admin only)
Code under test	server/src/routes/analytics.ts:15-40 GET /api/analytics (Admin Role Check)
Use case	Guarantees system analytics and operational metrics are protected behind admin-only role guards.
Preconditions	Users across all roles registered with auth tokens.
Steps	1. Send GET /api/analytics unauthenticated and verify 401 Unauthorized 2. Send GET /api/analytics as affected person and verify 403 Forbidden 3. Send GET /api/analytics as volunteer and verify 403 Forbidden 4. Send GET /api/analytics as admin and verify 200 OK with full analytics payload
Expected result	Non-admin callers rejected with 401/403; admin caller receives 200 OK with metrics.
SRS requirement(s)	FR-36, FR-4
Phase, last run	Phase 4; Pass (2026-11-01)

3.14 AI classification

Provides non-blocking natural language category and urgency suggestions via Gemini API with heuristic keyword fallbacks, ensuring zero disruption during network or service outages.

TC-AICL-01 should accurately classify medical emergencies with high/critical urgency using heuristic rules

Type, polarity, roles	Unit (Vitest); Positive; automated / internal
Test location	server/tests/classification.test.ts:34 Phase 4: AI & Heuristic Classification Service > should accurately classify medical emergencies with high/critical urgency using heuristic rules
Code under test	server/src/services/classification.ts:160-190 classifyHeuristic
Use case	Uses rule-based heuristic fallback to classify medical trauma and cardiac distress texts when AI network service is unavailable.
Preconditions	Classification service initialized.
Steps	1. Pass text “Elderly person having severe chest pain and difficulty breathing” to classifyHeuristic 2. Assert suggested category is “medical” and urgency is “high” or “critical”
Expected result	Identified as medical category with critical urgency rating.
SRS requirement(s)	FR-8
Phase, last run	Phase 4; Pass (2026-11-01)

TC-AICL-02 should accurately classify flood rescue emergencies with critical urgency

Type, polarity, roles	Unit (Vitest); Positive; automated / internal
Test location	server/tests/classification.test.ts:41 Phase 4: AI & Heuristic Classification Service > should accurately classify flood rescue emergencies with critical urgency
Code under test	server/src/services/classification.ts:160-190 classifyHeuristic
Use case	Detects rising floodwaters and trapped victims, categorizing them as rescue with critical urgency.
Preconditions	Classification service loaded.
Steps	1. Pass text “Family trapped on rooftop due to rising flood waters, need boat” to classifyHeuristic 2. Assert category is “rescue” and urgency is “critical”
Expected result	Identified as rescue category with critical urgency.
SRS requirement(s)	FR-8
Phase, last run	Phase 4; Pass (2026-11-01)

TC-AICL-03 should accurately classify shelter displacement requirements

Type, polarity, roles	Unit (Vitest); Positive; automated / internal
Test location	server/tests/classification.test.ts:48 Phase 4: AI & Heuristic Classification Service > should accurately classify shelter displacement requirements
Code under test	server/src/services/classification.ts:160-190 classifyHeuristic
Use case	Identifies displaced families lacking accommodation, categorizing them as shelter.
Preconditions	Classification service loaded.
Steps	1. Pass text “Our house collapsed in earthquake, 4 people sleeping outside in cold” to classifyHeuristic 2. Assert category is “shelter”
Expected result	Identified as shelter category.
SRS requirement(s)	FR-8
Phase, last run	Phase 4; Pass (2026-11-01)

TC-AICL-04 should accurately classify clean water and food supply requirements

Type, polarity, roles	Unit (Vitest); Positive; automated / internal
Test location	server/tests/classification.test.ts:53 Phase 4: AI & Heuristic Classification Service > should accurately classify clean water and food supply requirements
Code under test	server/src/services/classification.ts:160-190 classifyHeuristic
Use case	Identifies basic sustenance shortages, categorizing them as supplies or food.
Preconditions	Classification service loaded.
Steps	1. Pass text “Need drinking water and dry rations for 20 stranded villagers” to classifyHeuristic 2. Assert category is “supplies”
Expected result	Identified as supplies category.
SRS requirement(s)	FR-8
Phase, last run	Phase 4; Pass (2026-11-01)

TC-AICL-05 POST /api/classification/suggest: should return 200 with suggestion for valid emergency text

Type, polarity, roles	Integration (Vitest and Supertest); Positive; public / unauthenticated
Test location	server/tests/classification.test.ts:58 Phase 4: AI & Heuristic Classification Service > POST /api/classification/suggest: should return 200 with suggestion for valid emergency text
Code under test	server/src/routes/classification.ts:11-49 POST /api/classification/suggest
Use case	Provides real-time non-blocking category suggestions to citizens typing their emergency descriptions on the intake form.
Preconditions	Classification endpoint mounted.
Steps	1. Send POST /api/classification/suggest with text “Severe flooding in ground floor, family stranded” 2. Assert status 200 OK 3. Assert body contains suggestion object with category and urgency
Expected result	HTTP 200 OK with suggested category and urgency string.
SRS requirement(s)	FR-8
Phase, last run	Phase 4; Pass (2026-11-01)

TC-AICL-06 POST /api/classification/suggest: should return suggestion: null for empty or too short text

Type, polarity, roles	Integration (Vitest and Supertest); Positive; public / unauthenticated
Test location	server/tests/classification.test.ts:72 Phase 4: AI & Heuristic Classification Service > POST /api/classification/suggest: should return suggestion: null for empty or too short text
Code under test	server/src/routes/classification.ts:11-49 POST /api/classification/suggest
Use case	Returns null suggestion without executing AI or heuristic models when input text is fewer than minimum required characters.
Preconditions	Classification endpoint mounted.
Steps	1. Send POST /api/classification/suggest with text “help” 2. Assert status 200 OK 3. Assert body contains { suggestion: null }
Expected result	HTTP 200 OK with suggestion: null.
SRS requirement(s)	FR-8
Phase, last run	Phase 4; Pass (2026-11-01)

TC-AICL-07 POST /api/classification/suggest: should gracefully disable when ENABLE_AI_CLAS

Type, polarity, roles	Integration (Vitest and Supertest); Positive; public / unauthenticated
Test location	server/tests/classification.test.ts:82 Phase 4: AI & Heuristic Classification Service > POST /api/classification/suggest: should gracefully disable when ENABLE AI CLASSIFICATION=false
Code under test	server/src/routes/classification.ts:11-49 POST /api/classification/suggest
Use case	Allows operators to completely disable AI inference via environment variable without crashing the API or blocking requests.
Preconditions	Environment variable ENABLE AI CLASSIFICATION toggled to false.
Steps	1. Set ENABLE_AI_CLASSIFICATION=false 2. Send POST /api/classification/suggest with emergency text 3. Assert response status 200 OK with { suggestion: null, disabled: true }
Expected result	HTTP 200 OK with graceful disabled response payload.
SRS requirement(s)	FR-8
Phase, last run	Phase 4; Pass (2026-11-01)

TC-AICL-08 Decoupled guarantee: Request submission succeeds completely whether classification service is enabled or disabled

Type, polarity, roles	Integration (Vitest and Supertest); Positive; public / affected person
Test location	server/tests/classification.test.ts:95 Phase 4: AI & Heuristic Classification Service > Decoupled guarantee: Request submission succeeds completely whether classification service is enabled or disabled
Code under test	server/src/routes/requests.ts:104-179 POST /api/requests (Decoupled Resilient Submission)
Use case	Guarantees core emergency intake is fully decoupled and never fails even if the classification service crashes or is offline.
Preconditions	Emergency submission route mounted.
Steps	1. Disable classification service 2. Submit emergency ticket via POST /api/requests 3. Assert response status 201 Created and ticket is properly saved
Expected result	HTTP 201 Created; core disaster reporting succeeds unconditionally.
SRS requirement(s)	FR-8, FR-5
Phase, last run	Phase 4; Pass (2026-11-01)

3.15 Seed data and health

Verifies API service health check endpoints, Mongoose schema invariants and validators, and end-to-end database seed generation across all roles and resources.

TC-SEED-01 should return status 200 with *{ status: "ok" }*

Type, polarity, roles	Unit (Vitest); Positive; public / unauthenticated
Test location	server/tests/health.test.ts:6 GET /api/health > should return status 200 with *{* status: "ok" *}*
Code under test	server/src/app.ts:30-32 GET /api/health
Use case	Enables automated uptime monitoring and load-balancer health probes to verify API process readiness.
Preconditions	Express server running.
Steps	1. Send GET /api/health 2. Assert response status 200 OK 3. Assert JSON body equals *{* status: "ok" *}*
Expected result	HTTP 200 OK with status: "ok".
SRS requirement(s)	Supporting test (no SRS requirement)
Phase, last run	Phase 1; Pass (2026-11-01)

TC-SEED-02 should successfully seed sample users across all roles and sample requests

Type, polarity, roles	Integration (Vitest and Supertest); Positive; automated / admin
Test location	server/tests/models.test.ts:167 Seed Script > should successfully seed sample users across all roles and sample requests
Code under test	server/src/seed.ts:41-280 seedDatabase
Use case	Validates that the database seeding routine populates all user roles, categories, and initial crisis tickets without errors.
Preconditions	Clean test database connected.
Steps	1. Invoke seedDatabase function 2. Query User collection and verify count >= 5 across all roles 3. Query Request collection and verify seeded tickets exist in initial states
Expected result	All collections seeded with valid demo accounts and requests.
SRS requirement(s)	Supporting test (no SRS requirement)
Phase, last run	Phase 1; Pass (2026-11-01)

TC-SEED-03 should seed all collections with the rich expanded dataset

Type, polarity, roles	Integration (Vitest and Supertest); Positive; automated / admin
Test location	server/tests/seed.test.ts:38 Seed Script & End-to-End Phase 1 Integration Validation > should seed all collections with the rich expanded dataset
Code under test	server/src/seed.ts:41-280 seedDatabase (Expanded Seeding)
Use case	Verifies the expanded Phase 2-4 seeder populates organizations, inventory depots, categories, and incidents.
Preconditions	Clean test database.
Steps	1. Execute seedDatabase 2. Assert Organization, Resource, ResourceInventory, and EmergencyCategory collections are populated
Expected result	All expanded database collections populated with rich prototype data.
SRS requirement(s)	Supporting test (no SRS requirement)
Phase, last run	Phase 2; Pass (2026-11-01)

3.16 End-to-end scenarios

Executes multi-browser concurrent user journeys validating real-time synchronization, role-based UI action isolation, AI suggestion dismiss/apply, analytics dashboard filters, and text address geocoding.

TC-E2E-01 Full core scenario with independent concurrent browser contexts, real-time sync, comms isolation, and resolution confirmation

Type, polarity, roles	End-to-end (Playwright); Positive; affected person, admin, volunteer, resource provider
Test location	client/e2e/core-lifecycle-concurrency.spec.ts:4 ResQ Phase 5: Multi-Context Concurrent Core Crisis Lifecycle > Full core scenario with independent concurrent browser contexts, real-time sync, comms isolation, and resolution confirmation
Code under test	client/src/pages/SubmitRequestPage.tsx:1-542 SubmitRequestPage client/src/pages/DashboardPage.tsx:1-199 DashboardPage client/src/pages/TrackRequestPage.tsx:1-585 TrackRequestPage
Use case	Simulates real-world crisis operations across 4 concurrent browser contexts verifying live status changes, isolated chat threads, and citizen closure.
Preconditions	Backend API server and Vite client running; test database seeded.
Steps	1. Context A (Citizen): Navigates to /submit, waits for AI suggestion, and submits emergency request 2. Context B (Admin): Logs in, views dispatch console, and verifies request 3. Context C (Volunteer): Finds verified ticket, verifies matching panel, and claims ticket 4. Context A: Observes live transition to in progress without refreshing page 5. Context C & A: Exchange messages in request thread; Context D (Resource Provider) verifies thread isolation 6. Context C: Marks ticket as resolved 7. Context A: Sees live resolved state and clicks confirm resolution to reach closed
Expected result	Full concurrent lifecycle successfully completed with real-time updates and strict privacy isolation.
SRS requirement(s)	FR-1, FR-5, FR-6, FR-9, FR-12, FR-13, FR-14, FR-20, FR-27, FR-37
Phase, last run	Phase 5; Pass (2026-11-01)

TC-E2E-02 Phase 3B Permission Enforcement: Unauthorized roles never see forbidden actions

Type, polarity, roles	End-to-end (Playwright); Positive; resource provider, volunteer, admin
Test location	client/e2e/focused-edge-cases.spec.ts:8 ResQ Phase 5: Focused Edge-Case Coverage & Defect Prevention > Phase 3B Permission Enforcement: Unauthorized roles never see forbidden actions
Code under test	client/src/pages/DashboardPage.tsx:1-199 DashboardPage (Action Buttons & Guards)
Use case	Verifies the UI dynamically hides unauthorized action buttons: providers cannot claim tickets and admins cannot verify their own reports.
Preconditions	Seeded users with distinct roles logged into separate sessions.
Steps	1. Log in as resource_provider and confirm “Claim this request” button is absent 2. Log in as volunteer and confirm “Claim this request” button IS present 3. Log in as admin, submit emergency request, and verify UI disables self-verification
Expected result	UI strictly complies with Phase 3B RBAC rules hiding forbidden action controls.
SRS requirement(s)	FR-4, FR-12, FR-24, FR-37
Phase, last run	Phase 5; Pass (2026-11-01)

TC-E2E-03 Matching & Direct Invitation: Multi-factor breakdown and real in-app notification dispatch

Type, polarity, roles	End-to-end (Playwright); Positive; admin, volunteer
Test location	client/e2e/focused-edge-cases.spec.ts:114 ResQ Phase 5: Focused Edge-Case Coverage & Defect Prevention > Matching & Direct Invitation: Multi-factor breakdown and real in-app notification dispatch
Code under test	client/src/components/SuggestedMatchesPanel.tsx:1-249 SuggestedMatchesPanel client/src/components/NotificationList.tsx:1-320 NotificationList
Use case	Tests dispatch matching panel breakdown (proximity, suitability, capacity, availability) and verifies invite dispatch to responder.
Preconditions	Verified emergency ticket with candidate responder exists.
Steps	1. Admin opens verified ticket and inspects matching factor breakdown list 2. Admin clicks “Invite Responder” for top candidate 3. Candidate logs into independent session and verifies real in-app notification is received
Expected result	Matching factor elements rendered accurately; invitation notification received by responder.
SRS requirement(s)	FR-29, FR-31, FR-32, FR-37
Phase, last run	Phase 5; Pass (2026-11-01)

TC-E2E-04 AI Classification: Suggestion apply, dismiss, and uninterrupted submission during simulated outage

Type, polarity, roles	End-to-end (Playwright); Positive; affected person
Test location	client/e2e/focused-edge-cases.spec.ts:211 ResQ Phase 5: Focused Edge-Case Coverage & Defect Prevention > AI Classification: Suggestion apply, dismiss, and uninterrupted submission during simulated outage
Code under test	client/src/pages/SubmitRequestPage.tsx:1-542 SubmitRequestPage (AI Suggestion Integration)
Use case	Validates AI suggestion chip interaction (dismiss and apply) and proves emergency intake continues uninterrupted during AI service outages.
Preconditions	Submit page open in browser.
Steps	1. Type emergency description and observe AI suggestion chip appear 2. Click dismiss (x) and verify chip disappears 3. Retrigger suggestion and click apply, verifying category dropdown updates 4. Simulate 503 outage on classification API and submit request successfully
Expected result	Suggestion chip applies and dismisses smoothly; emergency request successfully submits during outage.
SRS requirement(s)	FR-8
Phase, last run	Phase 5; Pass (2026-11-01)

TC-E2E-05 Analytics Dashboard: Non-zero metrics readout and dynamic date-range filtering

Type, polarity, roles	End-to-end (Playwright); Positive; admin
Test location	client/e2e/focused-edge-cases.spec.ts:285 ResQ Phase 5: Focused Edge-Case Coverage & Defect Prevention > Analytics Dashboard: Non-zero metrics readout and dynamic date-range filtering
Code under test	client/src/pages/AnalyticsPage.tsx:1-540 AnalyticsPage
Use case	Validates admin analytics dashboard readout of Proposal Metrics (Time-to-Match, Double-Booking, Confirmation, Latency) and date range filtering.
Preconditions	Admin logged in; test tickets exist with lifecycle history.
Steps	1. Navigate to /analytics as admin 2. Confirm non-zero metric values for Time-to-Match and Double-Booking Rate 3. Toggle date range filter between “7 Days”, “Last 24 Hours”, and “All Time” 4. Confirm ticket sample count dynamically updates
Expected result	Analytics metrics render accurately and dynamic date filtering updates live readouts.
SRS requirement(s)	FR-36
Phase, last run	Phase 5; Pass (2026-11-01)

TC-E2E-06 Search address, pick from dropdown, submit request with privacy protection

Type, polarity, roles	End-to-end (Playwright); Positive; affected person
Test location	client/e2e/location-address-search.spec.ts:4 Additive Address Search on LocationPicker and Incident Management > Search address, pick from dropdown, submit request with privacy protection
Code under test	client/src/components/LocationPicker.tsx:1-310 LocationPicker (Address Search Input) client/src/pages/SubmitRequestPage.tsx:1-542 SubmitRequestPage
Use case	Verifies the text address search debounced dropdown on /submit moves the map pin and submits with proper privacy fuzzing.
Preconditions	Submit request page loaded.
Steps	1. Type address into search input on location step 2. Wait for debounced dropdown and select matching address candidate 3. Confirm map pin moves to selected coordinate 4. Submit emergency request and query API to confirm coordinates are fuzzed for public
Expected result	Pin moves accurately to searched address; public coordinates are obfuscated on save.
SRS requirement(s)	FR-5, FR-17
Phase, last run	Phase 5 (Location Address Search); Pass (2026-11-01)

TC-E2E-07 Incident creation address search recenters perimeter map

Type, polarity, roles	End-to-end (Playwright); Positive; admin
Test location	client/e2e/location-address-search.spec.ts:88 Additive Address Search on LocationPicker and Incident Management > Incident creation address search recenters perimeter map
Code under test	client/src/pages/AdminPage.tsx:1-170 AdminPage client/src/components/IncidentManagementPanel.tsx:1-991 IncidentManagementPanel (Address Search)
Use case	Tests text address search within the Incident Creation dialog, ensuring candidate selection recenters the disaster perimeter map.
Preconditions	Admin logged in on Incidents management page.
Steps	1. Open Incident creation modal 2. Type landmark into address search bar 3. Select candidate from dropdown 4. Verify incident perimeter map center shifts to target coordinates
Expected result	Perimeter map shifts to match selected address candidate.
SRS requirement(s)	FR-33
Phase, last run	Phase 5 (Location Address Search); Pass (2026-11-01)

4 Traceability

4.1 Endpoints to Test Cases

Endpoint	Tests	Test cases
GET /api/analytics	1	TC-ANLT-05
GET /api/auth/me	1	TC-AUTH-08
GET /api/categories	1	TC-CAT-01
GET /api/geocode	3	TC-PRIV-01, TC-PRIV-02, TC-PRIV-03
GET /api/health	1	TC-SEED-01
GET /api/incidents/:id/requests	1	TC-CAT-03
GET /api/inventory	1	TC-RESC-19
GET /api/notifications	2	TC-COMM-04, TC-COMM-06
GET /api/organizations	1	TC-ORG-05
GET /api/organizations/:id	1	TC-ORG-05
GET /api/requests	2	TC-PRIV-07, TC-LIFE-11
GET /api/requests/:id	4	TC-PRIV-08, TC-PRIV-09, TC-PRIV-10, TC-PRIV-11
GET /api/requests/:id/evidence	1	TC-EVID-01
GET /api/requests/:id/matches	1	TC-MTCH-05
GET /api/requests/:id/messages	3	TC-COMM-01, TC-COMM-02, TC-COMM-03
PATCH /api/categories/:id	1	TC-CAT-02
PATCH /api/notifications/:id/read	1	TC-COMM-06
PATCH /api/notifications/read-all	1	TC-COMM-06
PATCH /api/requests/:id/cancel	4	TC-CNCL-01, TC-CNCL-02, TC-CNCL-03, TC-CNCL-04
PATCH /api/requests/:id/claim	5	TC-RESC-01, TC-LIFE-01, TC-LIFE-02, TC-LIFE-03, TC-ORG-01
PATCH /api/requests/:id/confirm	3	TC-CNCL-05, TC-LIFE-01, TC-LIFE-05
PATCH /api/requests/:id/resolve	5	TC-CNCL-05, TC-LIFE-01, TC-LIFE-03, TC-LIFE-04, TC-ORG-01
PATCH /api/requests/:id/verify	8	TC-LIFE-01, TC-LIFE-04, TC-VRFY-01, TC-VRFY-02, TC-VRFY-03, TC-VRFY-04, TC-VRFY-05, TC-VRFY-06
POST /api/auth/login	3	TC-AUTH-05, TC-AUTH-06, TC-AUTH-07
POST /api/auth/register	4	TC-AUTH-01, TC-AUTH-02, TC-AUTH-03, TC-AUTH-04
POST /api/categories	1	TC-CAT-02
POST /api/classification/suggest	3	TC-AICL-05, TC-AICL-06, TC-AICL-07
POST /api/incidents	1	TC-CAT-03
POST /api/incidents/:id/requests	1	TC-CAT-03
POST /api/inventory/:id/release	2	TC-RESC-17, TC-RESC-18
POST /api/inventory/:id/reserve	3	TC-RESC-15, TC-RESC-16, TC-RESC-18
POST /api/organizations	4	TC-ORG-02, TC-ORG-03, TC-ORG-04, TC-ORG-06
POST /api/requests	7	TC-RESC-06, TC-AICL-08, TC-LIFE-08, TC-LIFE-09, TC-LIFE-10, TC-PRIV-06, TC-SOCK-08
POST /api/requests/:id/allocate	4	TC-RESC-02, TC-RESC-03, TC-RESC-04, TC-RESC-05
POST /api/requests/:id/evidence	3	TC-EVID-01, TC-EVID-02, TC-EVID-03
POST /api/requests/:id/invite	1	TC-MTCH-06
POST /api/requests/:id/messages	3	TC-COMM-01, TC-COMM-02, TC-COMM-03
POST /api/requests/:id/resources	5	TC-RESC-06, TC-RESC-07, TC-RESC-08, TC-RESC-09, TC-RESC-10
POST /api/resources	4	TC-RESC-11, TC-RESC-12, TC-RESC-13, TC-RESC-14

4.2 Roles to Permission Tests

Role	Allowed (positive tests)	Blocked (negative tests)
admin	TC-RESC-10, TC-ANLT-01, TC-ANLT-02, TC-ANLT-03, TC-ANLT-05, TC-AUTH-12, TC-CNCL-04, TC-CAT-02, TC-CAT-03, TC-LIFE-01, TC-MTCH-05, TC-MTCH-06, TC-COMM-01, TC-COMM-02, TC-SEED-02, TC-COMM-04, TC-PRIV-10, TC-SEED-03, TC-LIFE-12, TC-VRFY-03, TC-VRFY-04, TC-VRFY-05, TC-E2E-01, TC-E2E-02, TC-E2E-03, TC-E2E-05, TC-E2E-07	TC-RESC-05, TC-VRFY-06
volunteer	TC-RESC-01, TC-RESC-10, TC-ANLT-05, TC-AUTH-01, TC-AUTH-05, TC-AUTH-08, TC-CNCL-04, TC-CNCL-05, TC-CAT-02, TC-EVID-01, TC-LIFE-01, TC-LIFE-05, TC-MTCH-05, TC-MTCH-06, TC-COMM-01, TC-COMM-02, TC-COMM-04, TC-COMM-06, TC-VRFY-01, TC-PRIV-09, TC-PRIV-11, TC-LIFE-12, TC-SOCK-01, TC-SOCK-04, TC-SOCK-07, TC-SOCK-10, TC-E2E-01, TC-E2E-02, TC-E2E-03	TC-RESC-05, TC-AUTH-11, TC-EVID-03, TC-LIFE-02, TC-LIFE-03, TC-LIFE-04, TC-RESC-18
affected_person	TC-RESC-08, TC-ANLT-05, TC-CNCL-01, TC-CNCL-02, TC-CNCL-05, TC-AICL-08, TC-LIFE-01, TC-MTCH-05, TC-COMM-01, TC-COMM-02, TC-COMM-04, TC-LIFE-08, TC-LIFE-12, TC-SOCK-10, TC-E2E-01, TC-E2E-04, TC-E2E-06	TC-RESC-07, TC-LIFE-04, TC-LIFE-09, TC-LIFE-10, TC-PRIV-06, TC-RESC-18
resource_provider	TC-RESC-02, TC-ANLT-05, TC-RESC-11, TC-RESC-13, TC-RESC-15, TC-RESC-17, TC-SOCK-09, TC-E2E-01, TC-E2E-02	TC-RESC-03, TC-RESC-04, TC-RESC-12, TC-RESC-14, TC-RESC-16
ngo	TC-VRFY-01, TC-ORG-01, TC-ORG-02, TC-ORG-06	TC-VRFY-02, TC-ORG-03, TC-ORG-04
public / unauthenticated	TC-AICL-05, TC-AICL-06, TC-AICL-07, TC-PRIV-03, TC-SEED-01, TC-LIFE-06, TC-PRIV-07, TC-LIFE-11, TC-PRIV-08	TC-PRIV-01, TC-PRIV-02

4.3 Source Files to Test Cases

Source file	Tests	Test cases
client/src/components/IncidentManagementPanel.tsx	1	TC-E2E-07
client/src/components/LocationPicker.tsx	1	TC-E2E-06
client/src/components/NotificationList.tsx	1	TC-E2E-03
client/src/components/SuggestedMatchesPanel.tsx	1	TC-E2E-03
client/src/pages/AdminPage.tsx	1	TC-E2E-07
client/src/pages/AnalyticsPage.tsx	1	TC-E2E-05
client/src/pages/DashboardPage.tsx	2	TC-E2E-01, TC-E2E-02
client/src/pages/SubmitRequestPage.tsx	3	TC-E2E-01, TC-E2E-04, TC-E2E-06
client/src/pages/TrackRequestPage.tsx	1	TC-E2E-01
server/src/app.ts	1	TC-SEED-01
server/src/middleware/auth.ts	5	TC-AUTH-08, TC-AUTH-09, TC-AUTH-10, TC-AUTH-11, TC-AUTH-12
server/src/models/EmergencyCategory.ts	1	TC-CAT-01
server/src/models/Evidence.ts	1	TC-EVID-01
server/src/models/Incident.ts	1	TC-CAT-03
server/src/models/Message.ts	1	TC-COMM-01
server/src/models/Notification.ts	1	TC-COMM-04
server/src/models/Organization.ts	2	TC-ORG-02, TC-VRFY-01
server/src/models/Request.ts	4	TC-LIFE-06, TC-LIFE-07, TC-PRIV-04, TC-PRIV-05
server/src/models/RequestAssignment.ts	2	TC-CNCL-02, TC-RESC-01
server/src/models/RequestResource.ts	1	TC-RESC-02
server/src/models/Resource.ts	1	TC-RESC-11
server/src/models/ResourceInventory.ts	3	TC-CNCL-02, TC-RESC-11, TC-RESC-15
server/src/models/User.ts	4	TC-AUTH-01, TC-AUTH-13, TC-AUTH-14, TC-AUTH-15
server/src/models/Verification.ts	1	TC-VRFY-03
server/src/routes/analytics.ts	1	TC-ANLT-05
server/src/routes/auth.ts	8	TC-AUTH-01, TC-AUTH-02, TC-AUTH-03, TC-AUTH-04, TC-AUTH-05, TC-AUTH-06, TC-AUTH-07, TC-AUTH-08

Source file	Tests	Test cases
server/src/routes/categories.ts	2	TC-CAT-01, TC-CAT-02
server/src/routes/classification.ts	3	TC-AICL-05, TC-AICL-06, TC-AICL-07
server/src/routes/geocode.ts	3	TC-PRIV-01, TC-PRIV-02, TC-PRIV-03
server/src/routes/incidents.ts	1	TC-CAT-03
server/src/routes/inventory.ts	5	TC-RESC-15, TC-RESC-16, TC-RESC-17, TC-RESC-18, TC-RESC-19
server/src/routes/notifications.ts	2	TC-COMM-04, TC-COMM-06
server/src/routes/organizations.ts	5	TC-ORG-02, TC-ORG-03, TC-ORG-04, TC-ORG-05, TC-ORG-06
server/src/routes/requests.ts	49	TC-AICL-08, TC-CNCL-01, TC-CNCL-02, TC-CNCL-03, TC-CNCL-04, TC-CNCL-05, TC-COMM-01, TC-COMM-02, TC-COMM-03, TC-COMM-05, TC-EVID-01, TC-EVID-02, TC-EVID-03, TC-LIFE-01, TC-LIFE-02, TC-LIFE-03, TC-LIFE-04, TC-LIFE-05, TC-LIFE-08, TC-LIFE-09, TC-LIFE-10, TC-LIFE-11, TC-LIFE-12, TC-MTCH-05, TC-MTCH-06, TC-ORG-01, TC-PRIV-06, TC-PRIV-07, TC-PRIV-08, TC-PRIV-09, TC-PRIV-10, TC-PRIV-11, TC-RESC-01, TC-RESC-02, TC-RESC-03, TC-RESC-04, TC-RESC-05, TC-RESC-06, TC-RESC-07, TC-RESC-08, TC-RESC-09, TC-RESC-10, TC-SOCK-08, TC-VRFY-01, TC-VRFY-02, TC-VRFY-03, TC-VRFY-04, TC-VRFY-05, TC-VRFY-06
server/src/routes/resources.ts	4	TC-RESC-11, TC-RESC-12, TC-RESC-13, TC-RESC-14
server/src/seed.ts	2	TC-SEED-02, TC-SEED-03
server/src/services/analytics.ts	3	TC-ANLT-01, TC-ANLT-02, TC-ANLT-03
server/src/services/classification.ts	4	TC-AICL-01, TC-AICL-02, TC-AICL-03, TC-AICL-04
server/src/services/matching.ts	4	TC-MTCH-01, TC-MTCH-02, TC-MTCH-03, TC-MTCH-04
server/src/socket/index.ts	11	TC-ANLT-04, TC-SOCK-01, TC-SOCK-02, TC-SOCK-03, TC-SOCK-04, TC-SOCK-05, TC-SOCK-06, TC-SOCK-07, TC-SOCK-08, TC-SOCK-09, TC-SOCK-10
server/src/utils/sanitize.ts	4	TC-PRIV-07, TC-PRIV-08, TC-PRIV-09, TC-SOCK-06

4.4 SRS Functional Requirements to Test Cases

The table shows which test cases support each functional requirement of the Software Requirements Specification. The mapping was prepared by the authors.

ID	Requirement	Tests	Test cases
FR-1	Anonymous submission and tracking	5	TC-E2E-01, TC-LIFE-06, TC-LIFE-08, TC-RESC-06, TC-SOCK-02
FR-2	Self-registration (no administrator self-registration)	7	TC-AUTH-01, TC-AUTH-02, TC-AUTH-03, TC-AUTH-04, TC-AUTH-13, TC-AUTH-14, TC-AUTH-15
FR-3	Email and password login, bcrypt hash, JWT	7	TC-AUTH-01, TC-AUTH-05, TC-AUTH-06, TC-AUTH-07, TC-AUTH-08, TC-AUTH-09, TC-AUTH-10
FR-4	Server-side role checks	11	TC-ANLT-05, TC-AUTH-11, TC-AUTH-12, TC-AUTH-14, TC-CAT-02, TC-CNCL-04, TC-E2E-02, TC-LIFE-04, TC-MTCH-05, TC-RESC-05, TC-RESC-18
FR-5	Request fields and location (pin or address search)	11	TC-AICL-08, TC-E2E-01, TC-E2E-06, TC-LIFE-06, TC-LIFE-08, TC-LIFE-09, TC-PRIV-01, TC-PRIV-02, TC-PRIV-03, TC-PRIV-06, TC-RESC-06
FR-6	Tracking identifier REQ-YYYY-XXXXXX	4	TC-E2E-01, TC-LIFE-08, TC-LIFE-12, TC-PRIV-04
FR-7	Only active categories offered	3	TC-CAT-01, TC-LIFE-07, TC-LIFE-10
FR-8	Optional category and urgency suggestion	9	TC-AICL-01, TC-AICL-02, TC-AICL-03, TC-AICL-04, TC-AICL-05, TC-AICL-06, TC-AICL-07, TC-AICL-08, TC-E2E-04
FR-9	Statuses and forward-only lifecycle	5	TC-E2E-01, TC-LIFE-01, TC-LIFE-02, TC-LIFE-03, TC-LIFE-12
FR-10	Verification by administrator or verified NGO; duplicate or unfounded closed	6	TC-LIFE-01, TC-LIFE-04, TC-VRFY-01, TC-VRFY-03, TC-VRFY-04, TC-VRFY-05
FR-11	No self or same-organisation verification	2	TC-VRFY-02, TC-VRFY-06
FR-12	Claim by volunteers and NGO members, verified requests only	6	TC-E2E-01, TC-E2E-02, TC-LIFE-01, TC-LIFE-04, TC-LIFE-12, TC-RESC-01
FR-13	Resolve by claimant, administrator or claiming organisation	5	TC-CNCL-05, TC-E2E-01, TC-LIFE-01, TC-LIFE-12, TC-ORG-01

ID	Requirement	Tests	Test cases
FR-14	Close only by requester confirmation	6	TC-CNCL-05, TC-E2E-01, TC-LIFE-01, TC-LIFE-05, TC-LIFE-12, TC-VRFY-05
FR-15	Withdrawal by requester or administrator	4	TC-CNCL-01, TC-CNCL-02, TC-CNCL-03, TC-CNCL-04
FR-16	Status history entry on every change	2	TC-LIFE-01, TC-VRFY-03
FR-17	Exact and coarse public location (200 to 400 m)	3	TC-E2E-06, TC-PRIV-04, TC-PRIV-05
FR-18	Exact location and phone only to authorised users	5	TC-PRIV-07, TC-PRIV-08, TC-PRIV-09, TC-PRIV-10, TC-PRIV-11
FR-19	Same redaction on real-time data	2	TC-SOCK-06, TC-SOCK-07
FR-20	Real-time push after the database write	11	TC-E2E-01, TC-SOCK-01, TC-SOCK-02, TC-SOCK-03, TC-SOCK-04, TC-SOCK-05, TC-SOCK-06, TC-SOCK-07, TC-SOCK-08, TC-SOCK-09, TC-SOCK-10
FR-21	Resynchronise after reconnect	0	*No automated test*
FR-22	Register resources and inventory for own organisation	5	TC-RESC-11, TC-RESC-12, TC-RESC-13, TC-RESC-14, TC-RESC-19
FR-23	Supply needs added only by requester, administrator or claimant	4	TC-RESC-07, TC-RESC-08, TC-RESC-09, TC-RESC-10
FR-24	Allocation by NGO or resource provider within real stock	5	TC-E2E-02, TC-RESC-02, TC-RESC-03, TC-RESC-04, TC-RESC-05
FR-25	Reserve and release restricted by role	5	TC-CNCL-02, TC-RESC-15, TC-RESC-16, TC-RESC-17, TC-RESC-18
FR-26	Fulfilment shown for each item	1	TC-RESC-02
FR-27	Request-scoped messaging rules	6	TC-COMM-01, TC-COMM-02, TC-COMM-03, TC-E2E-01, TC-ORG-01, TC-SOCK-10
FR-28	Phone number unlocks thread for anonymous requester	1	TC-COMM-03
FR-29	In-app notifications	4	TC-COMM-04, TC-COMM-05, TC-COMM-06, TC-E2E-03
FR-30	Evidence attachments	3	TC-EVID-01, TC-EVID-02, TC-EVID-03
FR-31	Ranked matching suggestions	6	TC-E2E-03, TC-MTCH-01, TC-MTCH-02, TC-MTCH-03, TC-MTCH-04, TC-MTCH-05
FR-32	Invitation by notification	2	TC-E2E-03, TC-MTCH-06
FR-33	Incidents	2	TC-CAT-03, TC-E2E-07
FR-34	Emergency categories	2	TC-CAT-01, TC-CAT-02
FR-35	Organisation directory and profile	5	TC-ORG-02, TC-ORG-03, TC-ORG-04, TC-ORG-05, TC-ORG-06
FR-36	Analytics dashboard	6	TC-ANLT-01, TC-ANLT-02, TC-ANLT-03, TC-ANLT-04, TC-ANLT-05, TC-E2E-05
FR-37	Console with map and list	4	TC-E2E-01, TC-E2E-02, TC-E2E-03, TC-LIFE-11
FR-38	Public home page	0	*No automated test*

Requirements with no automated test: FR-21 (resynchronise after reconnect), FR-38 (public home page). Seed data and health-check tests support the system but do not map to a single requirement.

5 Manual Validation Records

The pilot was run on 20 September 2026 by a single operator who used all five roles in separate browser sessions, not by independent participants. All records in this chapter are manual and are not part of the automated suite. The pilot substitutes for, but does not fully replicate, the independent multi-user pilot described in the original proposal.

5.1 Tickets Processed

Ticket	Category	Final status	Verification	Claimed by	Confirmation
REQ-2026-382977	medical	resolved	Admin Phone Call (High priority)	Rahul Sharma (Volunteer)	Pending
REQ-2026-535413	medical	closed	Admin Phone Call (High priority)	Rahul Sharma (Volunteer)	Confirmed Closed
REQ-2026-123714	supplies	resolved	Admin Phone Call (Medium priority)	Punjab Relief Network (NGO)	Pending
REQ-2026-860087	medical	resolved	Admin Phone Call (High priority)	Rahul Sharma (Volunteer)	Pending

5.2 Simultaneous-Claim Test

One verified ticket was claimed from two windows at almost the same moment, by a volunteer and by an NGO member, about one second apart. The volunteer's claim succeeded and the ticket moved to in progress. The NGO member's claim was rejected immediately with an HTTP 409 conflict response and an on-screen message. The exported data contains exactly one assignment record for the ticket, so no double booking occurred. This is a manual check of the same rule that the automated tests verify.

5.3 Measured Metrics

The targets are those set in the original project proposal.

Metric	Target	Measured	Note
Time-to-Match	Median at most 15 minutes	1.0 minute median (4 tickets)	Measures the system overhead for one operator who knew the script. It does not predict real dispatch time.
Double-booking rate	0%	0.0% (0 conflicts in 4 assignments)	Also checked by the simultaneous-claim test above.
Confirmation rate	No numeric target in the proposal	25.0% (1 of 4 resolved tickets confirmed)	Reported without a pass or fail verdict. Three citizens had not confirmed by the end of the session.
Propagation latency	Under 2 seconds	23 ms median, 29 ms at the 95th percentile (8 samples)	Measured on one machine, so network delay is not included.

6 Known Coverage Gaps

The route handlers and role guards in the server were compared with what the automated tests call. The following gaps remain. They are reported so that the coverage claimed above is not overstated.

6.1 Route Endpoints With No Automated Tests

These endpoints are implemented in the Express router but have no dedicated Supertest integration test:

- GET /api/inventory/:id: Retrieve single depot inventory record by ID.
- GET /api/resources/:id: Retrieve catalog resource definition by ID.
- GET /api/incidents: List all major disaster incidents (incident creation and ticket attachments are tested, but the plain list endpoint is not).
- GET /api/incidents/:id: Retrieve incident perimeter metadata by ID.
- GET /api/requests/:id/verification: Retrieve the specific Verification audit document for a request.
- GET /api/requests/:id/resources: List itemized resource requirements (tested via POST append, but standalone GET query lacks a dedicated test).
- GET /api/requests/:id/allocation: Retrieve allocated inventory breakdown (tested via POST allocate, but standalone GET query lacks a dedicated test).

6.2 Endpoints With Happy-Path Only (Missing Negative Tests)

These endpoints have working positive integration tests but lack negative boundary or rejection tests:

- POST /api/categories: Tested for admin creation, but lacks tests for non-admin rejection (403) or duplicate category name (409).
- PATCH /api/categories/:id: Tested for admin category update, but lacks tests for non-admin rejection or invalid category ID.
- POST /api/incidents: Tested for admin creation, but lacks negative tests for missing radius / coordinates.
- POST /api/incidents/:id/requests: Tested for valid ticket attachment, but lacks negative tests for attaching non-existent request IDs.
- PATCH /api/requests/:id/confirm: Tested for valid citizen confirmation, but lacks negative tests asserting that a stranger cannot confirm someone else's ticket.
- PATCH /api/requests/:id/cancel: Tested for cancellation flow, but lacks negative test asserting 400 rejection when attempting to cancel an already closed request.
- PATCH /api/notifications/:id/read: Tested for marking read, but lacks negative tests asserting that User B cannot mark User A's notification as read.
- PATCH /api/notifications/read-all: Tested for authenticated user, but lacks unauthenticated rejection test.
- GET /api/requests/:id/evidence: Tested for evidence listing, but lacks negative tests for requests in private unassigned states.
- POST /api/requests/:id/invite: Tested for valid dispatcher invite, but lacks negative test for inviting responders to an unverified ticket.